



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Lógica Digital

Circuitos secuenciales: Máquinas de estado

Sistemas Digitales

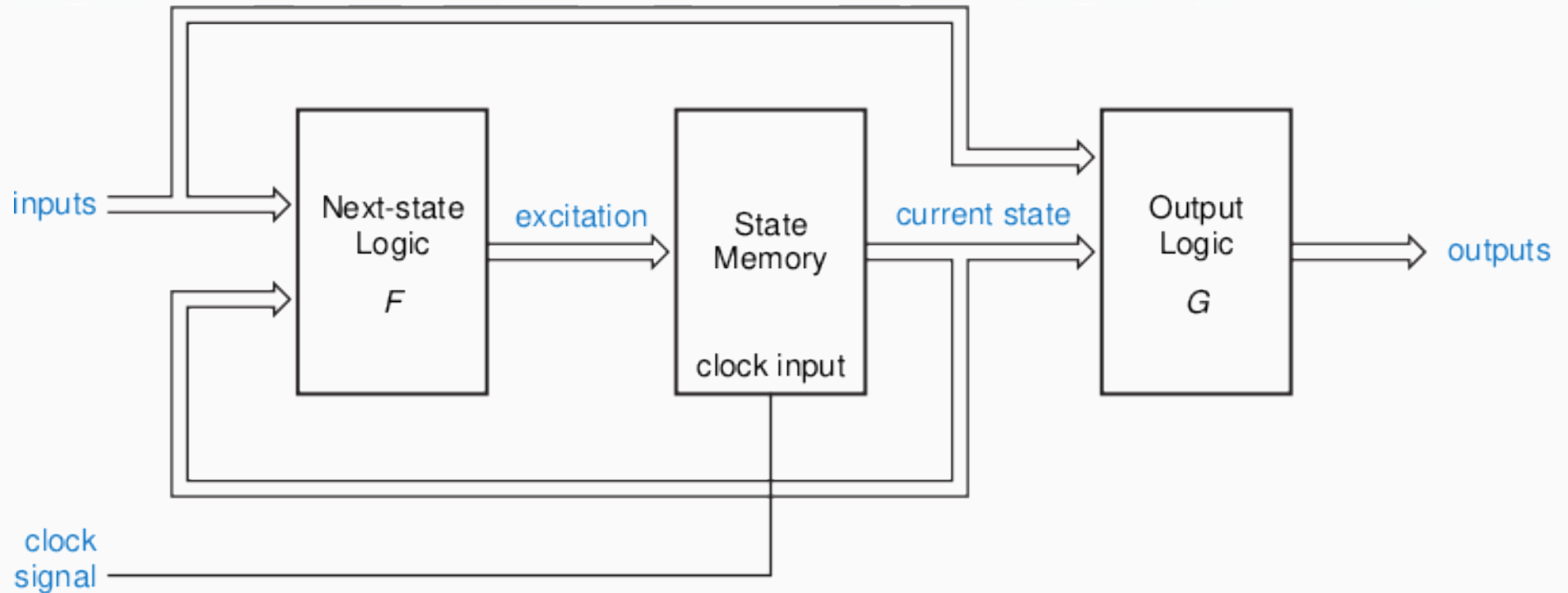
2do Cuatrimestre 2026

FCEN - UBA

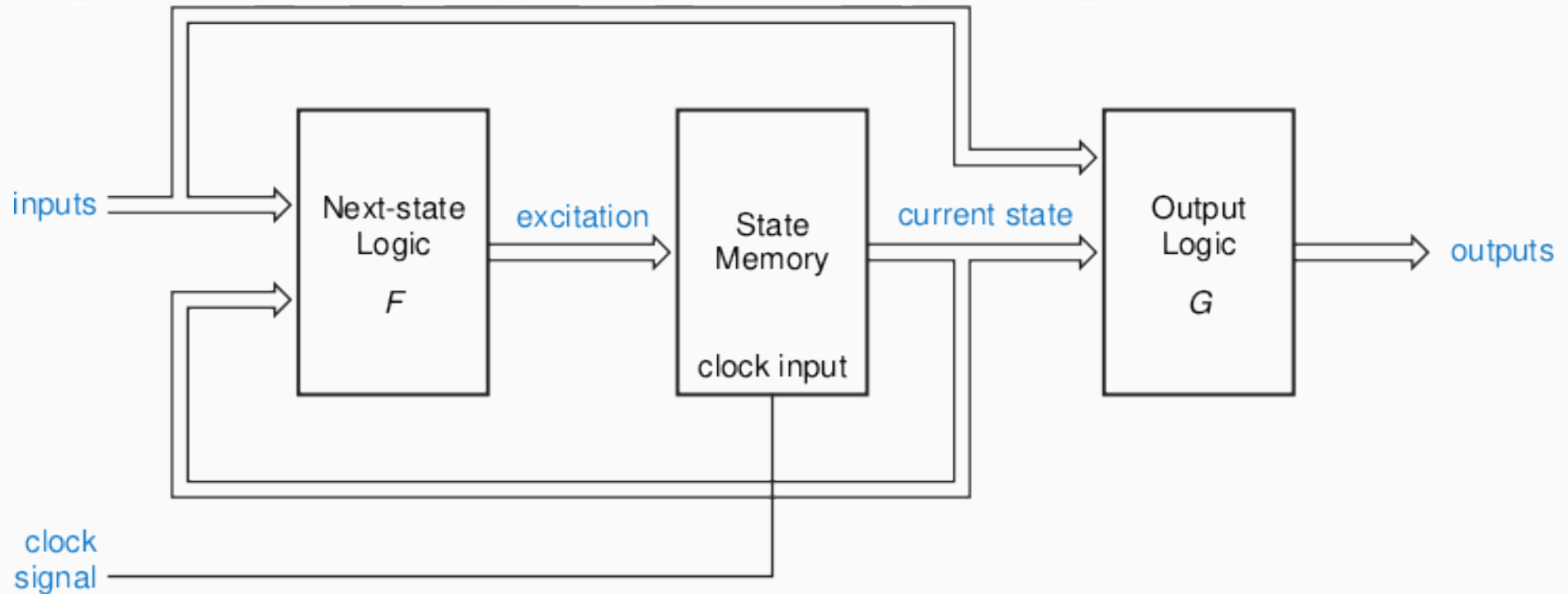
Máquinas de estado

Modelo general de un circuito secuencial

Conceptualmente podemos pensar a un sistema secuencial como:

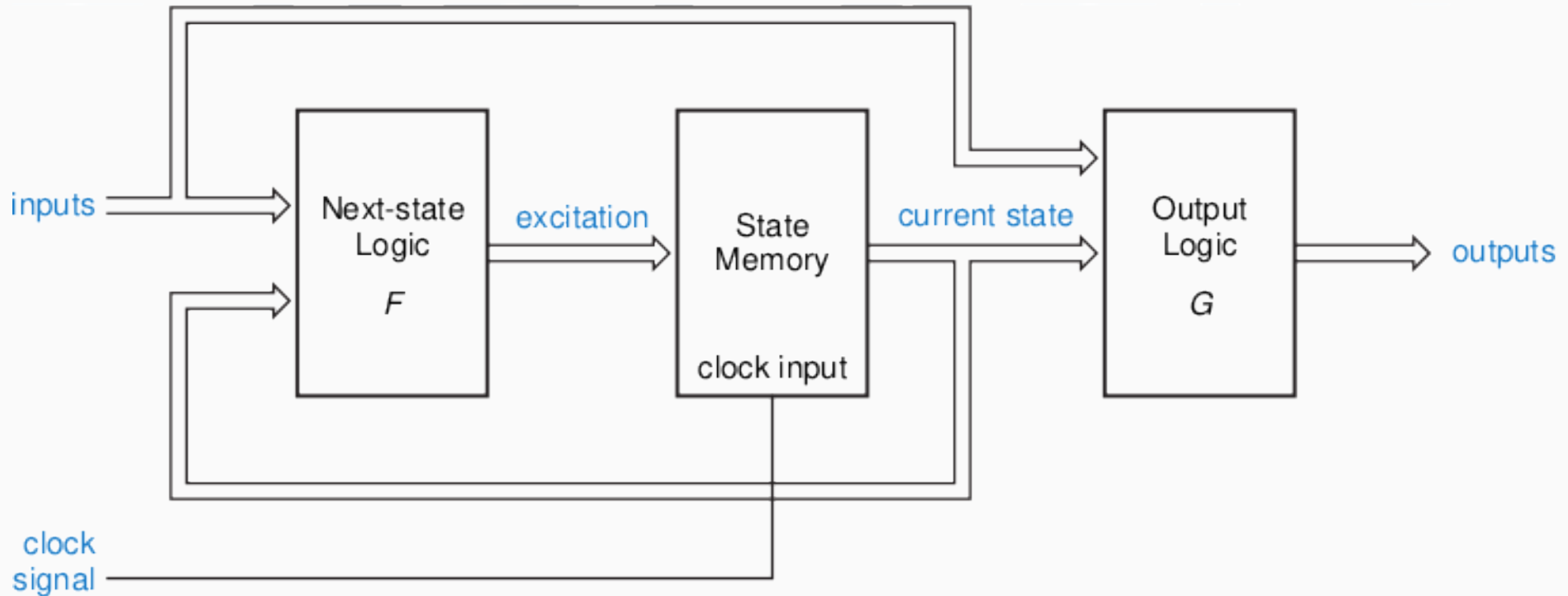


Conceptualmente podemos pensar a un sistema secuencial como:



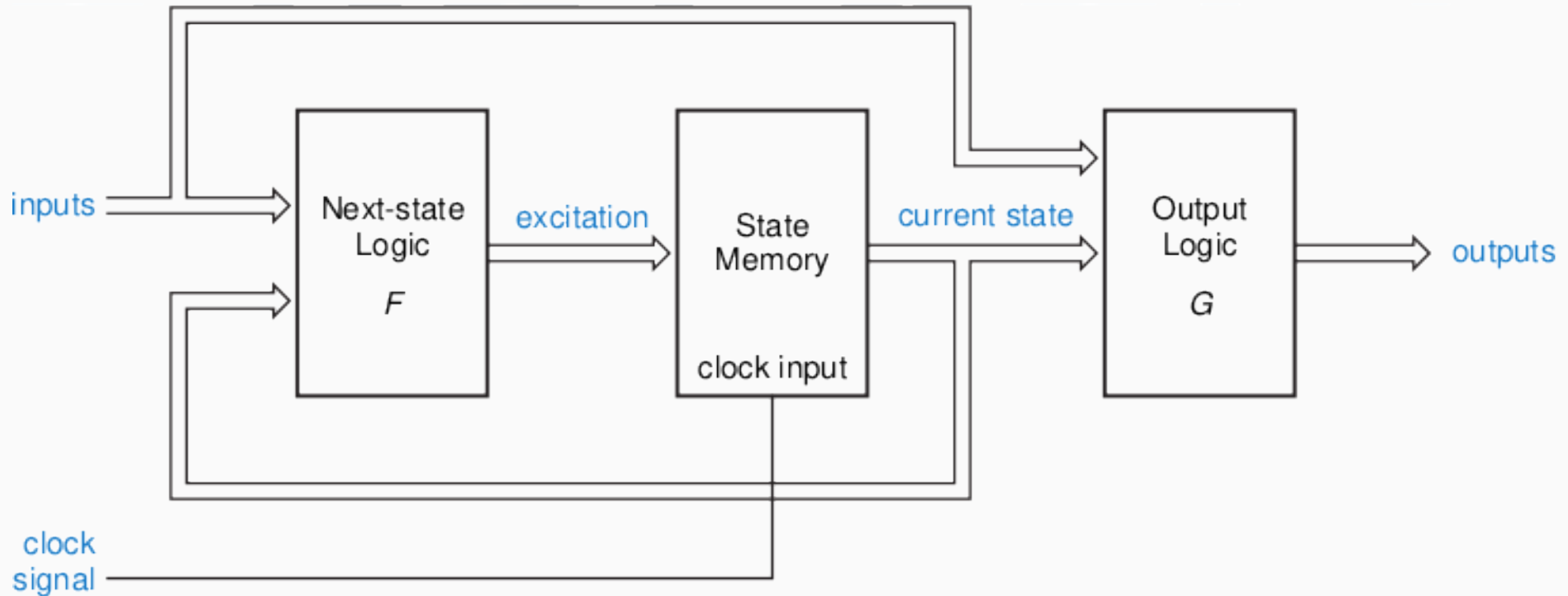
- **Lógica de próximo estado:** $F(\text{estado}_{\text{actual}}, \text{entradas})$

Conceptualmente podemos pensar a un sistema secuencial como:



- **Lógica de próximo estado:** $F(\text{estado}_{\text{actual}}, \text{entradas})$
- **Registro de estado:** $\text{estado}_{\text{actual}} \xleftarrow{\text{clock}} \text{estado}_{\text{siguiente}}$

Conceptualmente podemos pensar a un sistema secuencial como:



- **Lógica de próximo estado:** $F(\text{estado}_{\text{actual}}, \text{entradas})$
- **Registro de estado:** $\text{estado}_{\text{actual}} \xleftarrow{\text{clock}} \text{estado}_{\text{siguiente}}$
- **Lógica de salida:** $G(\text{estado}_{\text{actual}}, \text{entradas})$

⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).

¹Con un registro de estado de N bits se pueden representar 2^N estados.

- ⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).
- ⇐ Con FSMs podemos modelar cualquier sistema que pueda ser representado por un número finito de estados.

¹Con un registro de estado de N bits se pueden representar 2^N estados.

⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).

⇐ Con FSMs podemos modelar cualquier sistema que pueda ser representado por un número finito de estados.

Una máquina de estados queda definida por:

¹Con un registro de estado de N bits se pueden representar 2^N estados.

⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).

⇐ Con FSMs podemos modelar cualquier sistema que pueda ser representado por un número finito de estados.

Una máquina de estados queda definida por:

- Un conjunto finito de estados

¹Con un registro de estado de N bits se pueden representar 2^N estados.

⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).

⇐ Con FSMs podemos modelar cualquier sistema que pueda ser representado por un número finito de estados.

Una máquina de estados queda definida por:

- Un conjunto finito de estados
- Un conjunto finito de entradas y salidas

¹Con un registro de estado de N bits se pueden representar 2^N estados.

⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).

⇐ Con FSMs podemos modelar cualquier sistema que pueda ser representado por un número finito de estados.

Una máquina de estados queda definida por:

- Un conjunto finito de estados
- Un conjunto finito de entradas y salidas
- Una función de transición de estados

¹Con un registro de estado de N bits se pueden representar 2^N estados.

⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).

⇐ Con FSMs podemos modelar cualquier sistema que pueda ser representado por un número finito de estados.

Una máquina de estados queda definida por:

- Un conjunto finito de estados
- Un conjunto finito de entradas y salidas
- Una función de transición de estados
- Una función de salida

¹Con un registro de estado de N bits se pueden representar 2^N estados.

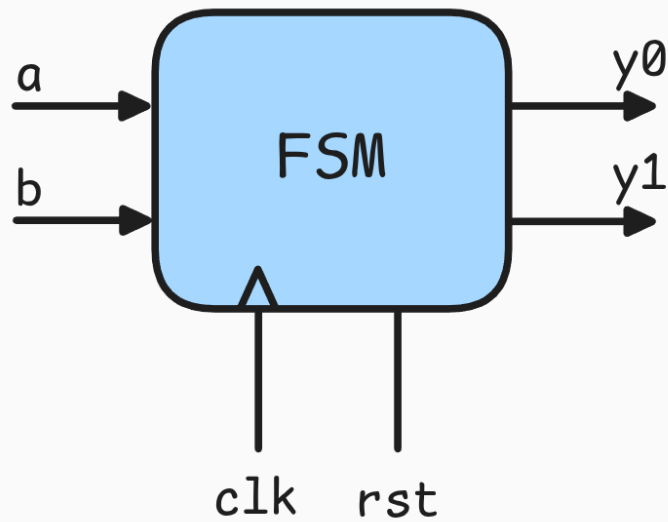
⇒ Los circuitos secuenciales pueden ser pensados formalmente como una *Máquina de Estados Finitos*¹ o *FSM* (del inglés *Finite State Machine*).

⇐ Con FSMs podemos modelar cualquier sistema que pueda ser representado por un número finito de estados.

Una máquina de estados queda definida por:

- Un conjunto finito de estados
- Un conjunto finito de entradas y salidas
- Una función de transición de estados
- Una función de salida
- Un estado inicial

¹Con un registro de estado de N bits se pueden representar 2^N estados.



Interfaz

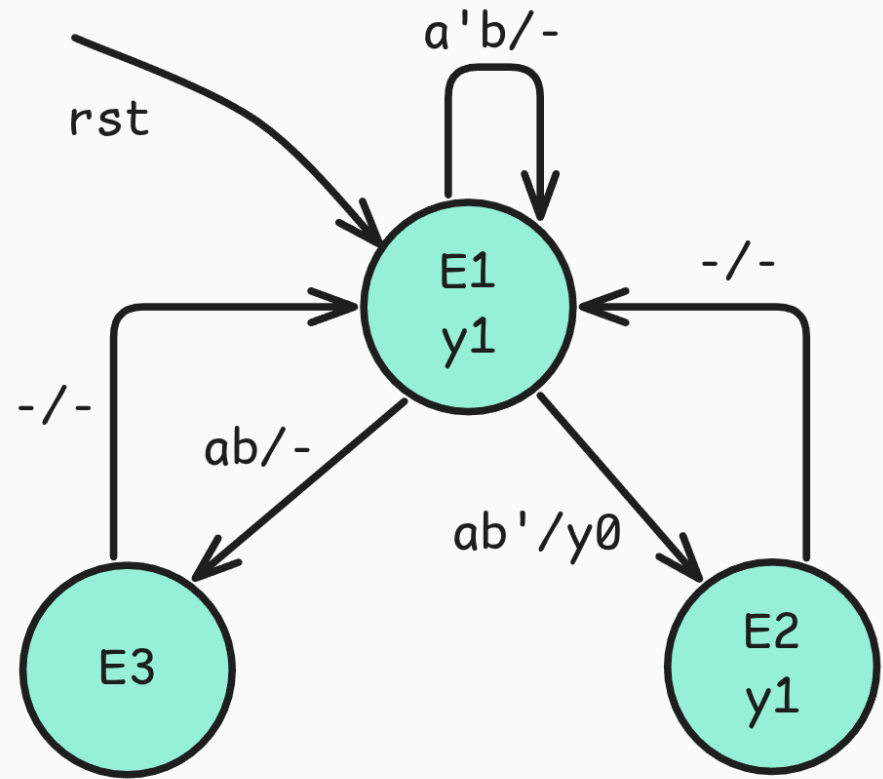
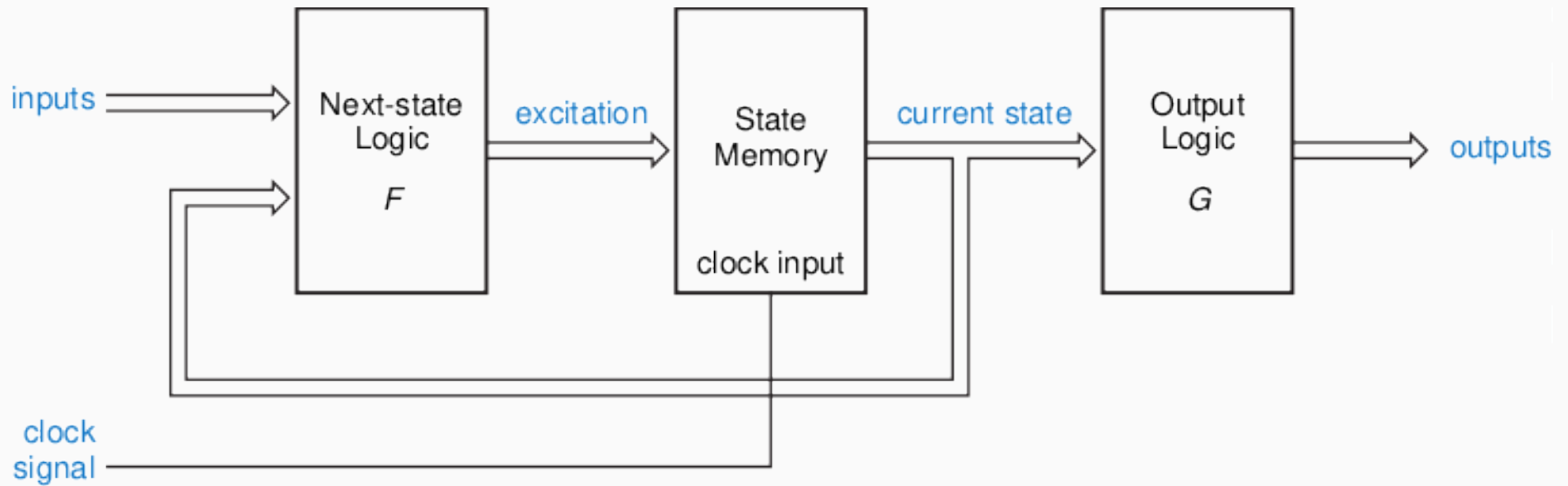
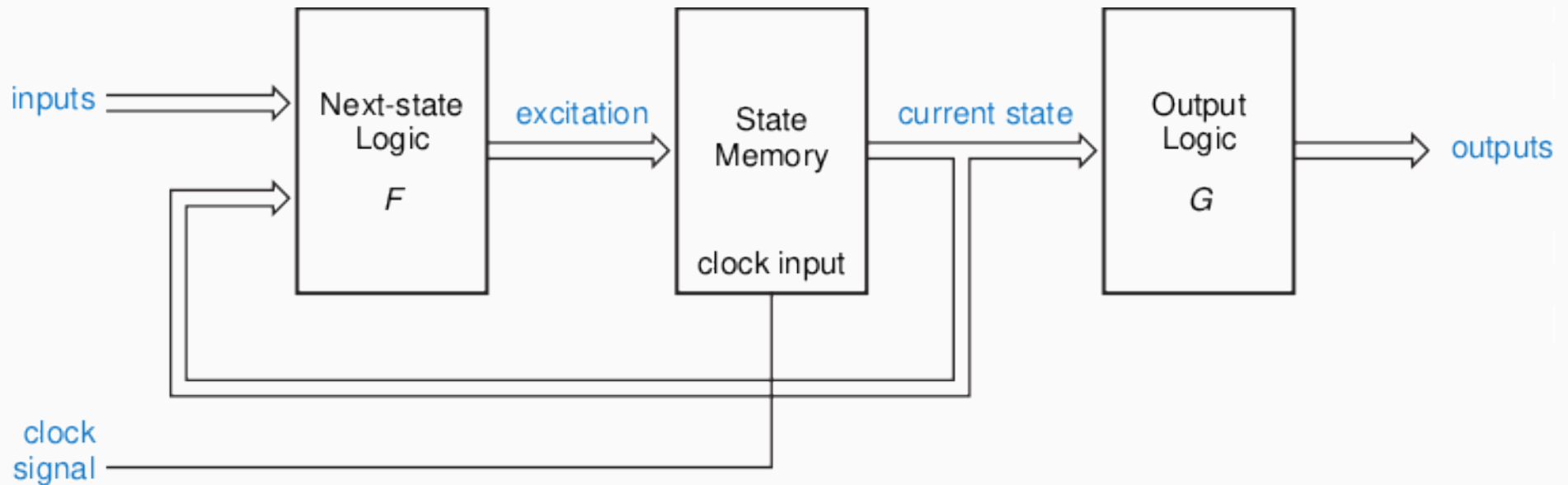


Diagrama de estados

Todas las salidas en $\emptyset$ salvo que se explicita lo contrario

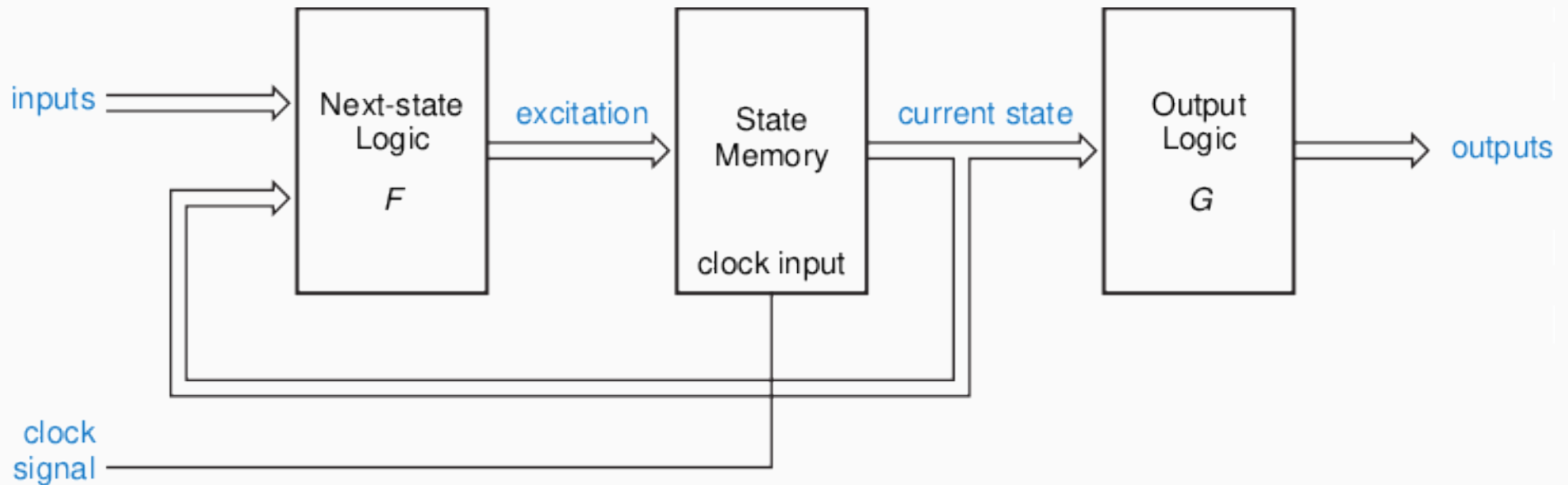


Características:



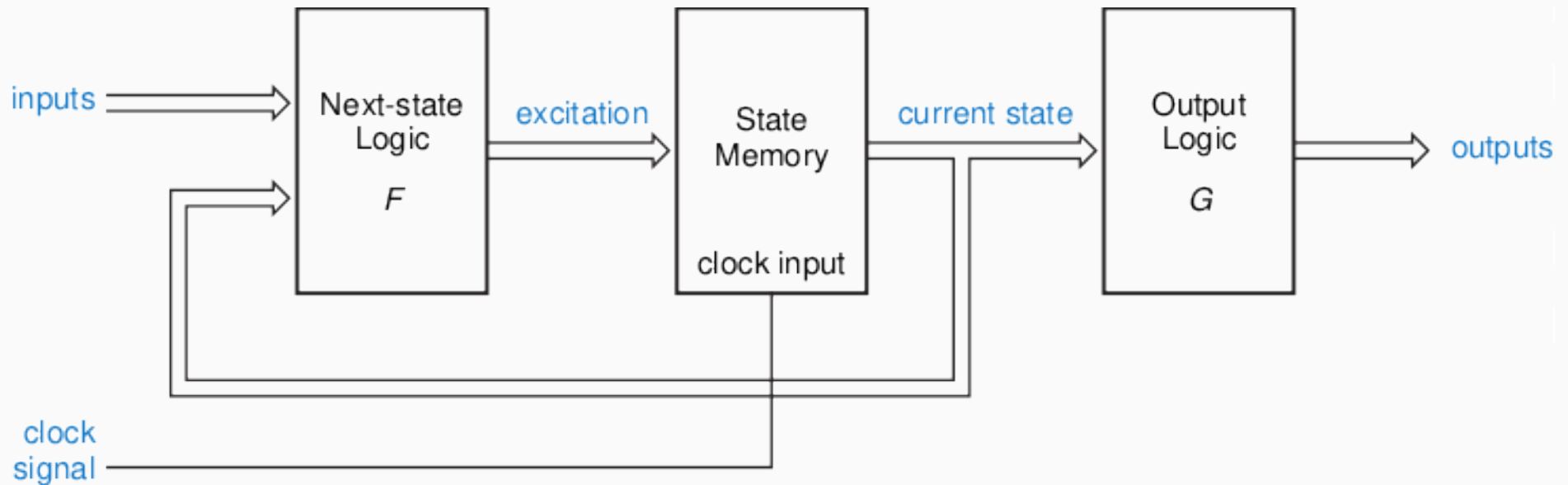
Características:

- La salida siempre cambia un clock después que se dispara la condición de transición



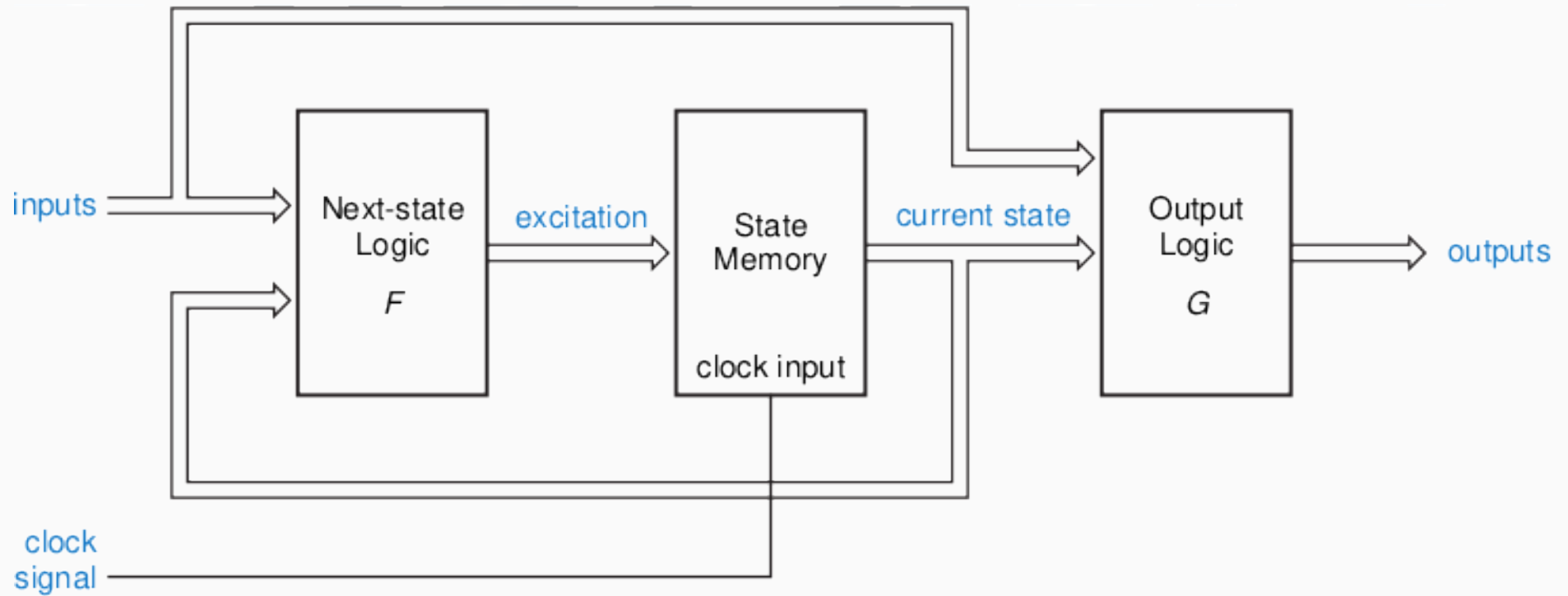
Características:

- La salida siempre cambia un clock después que se dispara la condición de transición
- No produce *glitches* a la salida

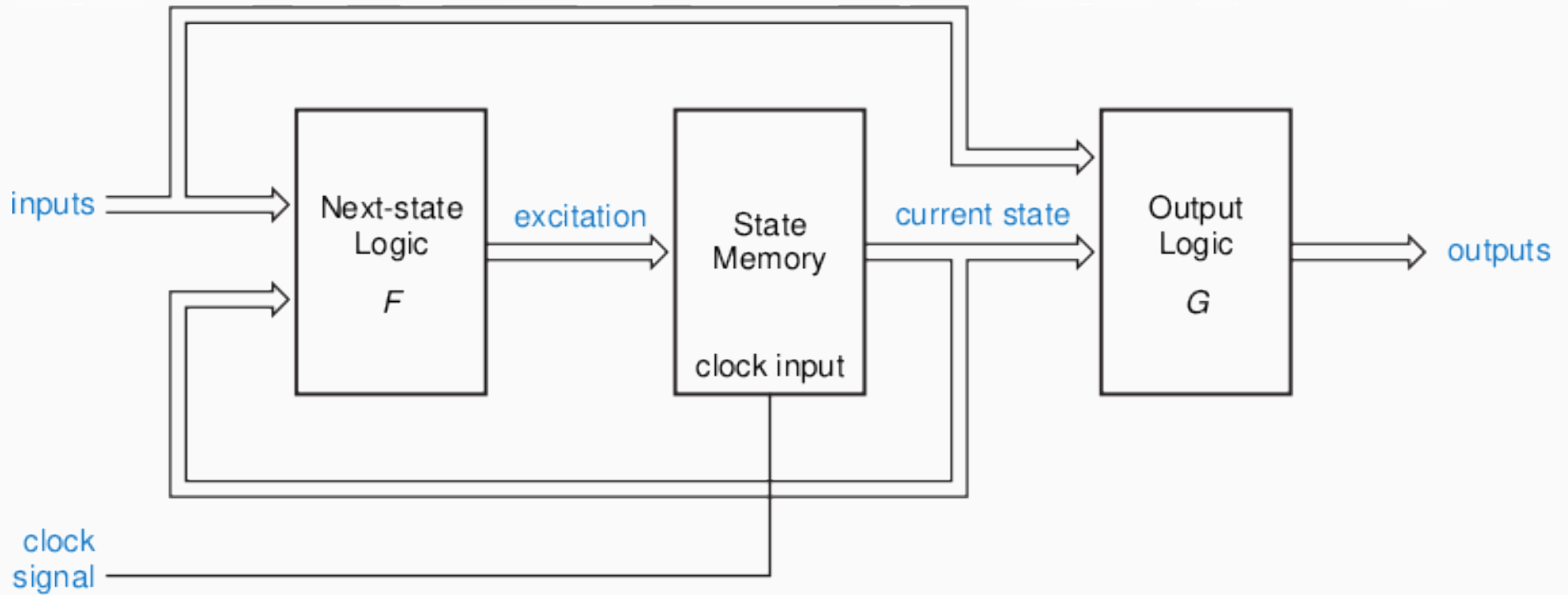


Características:

- La salida siempre cambia un clock después que se dispara la condición de transición
- No produce *glitches* a la salida
- La cantidad de estados para reproducir cierto comportamiento puede ser más grande que con *Mealy*

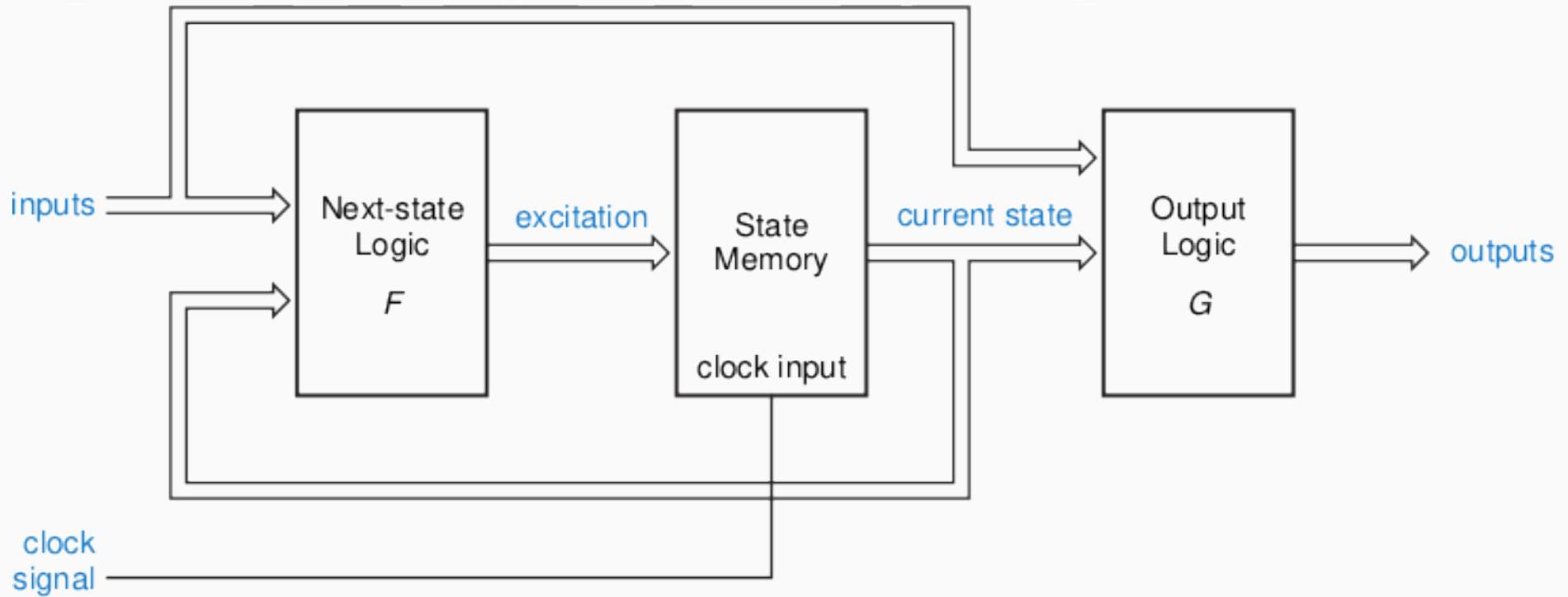


Características:



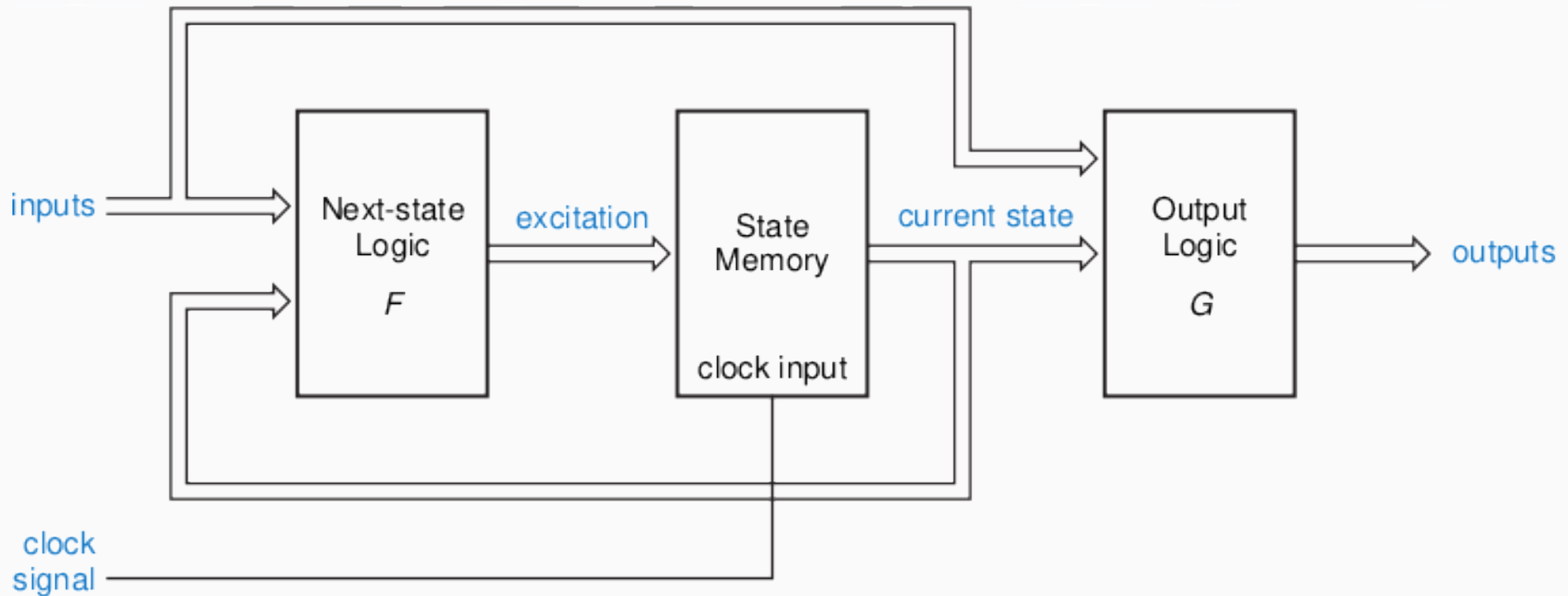
Características:

- La salida puede cambiar dentro del mismo clock en que se dispara la condición de transición



Características:

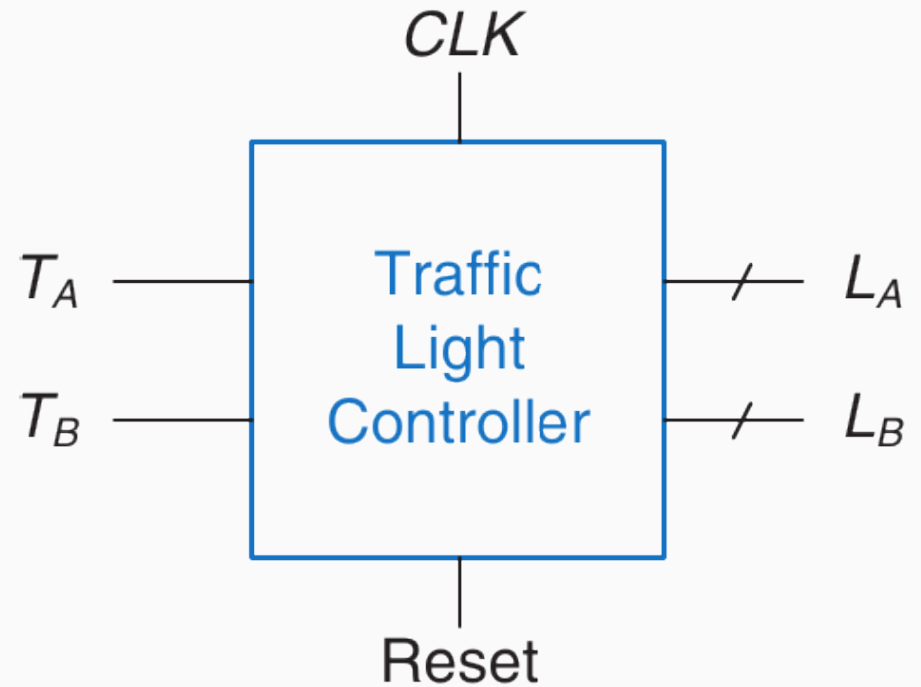
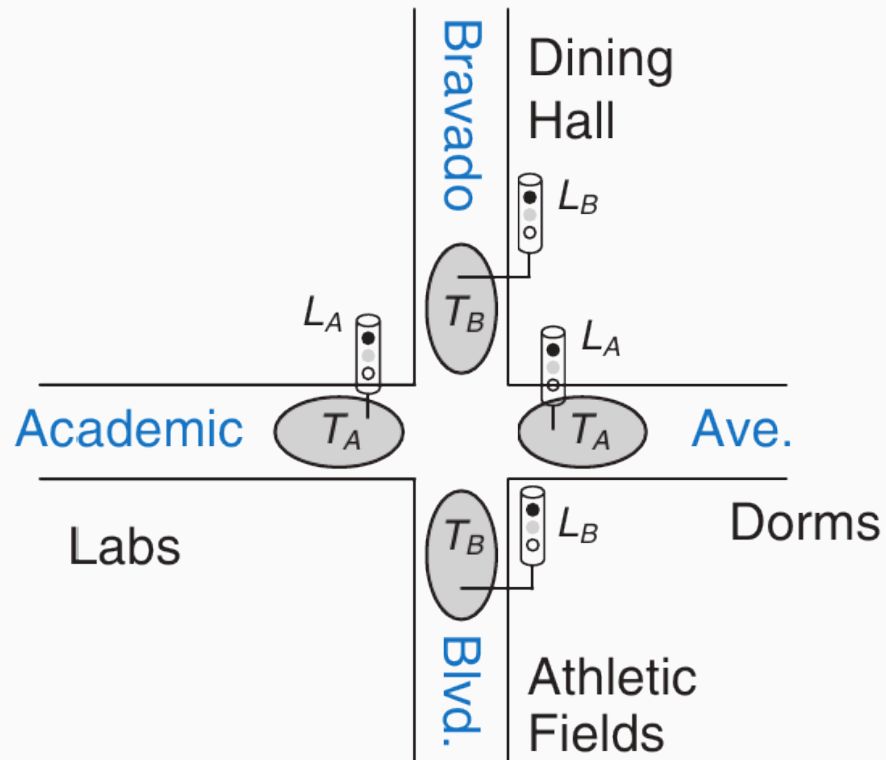
- La salida puede cambiar dentro del mismo clock en que se dispara la condición de transición
- Produce *glitches* a la salida



Características:

- La salida puede cambiar dentro del mismo clock en que se dispara la condición de transición
- Produce *glitches* a la salida
- La cantidad de estados para reproducir cierto comportamiento generalmente es más chica que en *Moore*

Ejemplo: Controlador de luces de tráfico



Ejemplo: Controlador de luces de tráfico

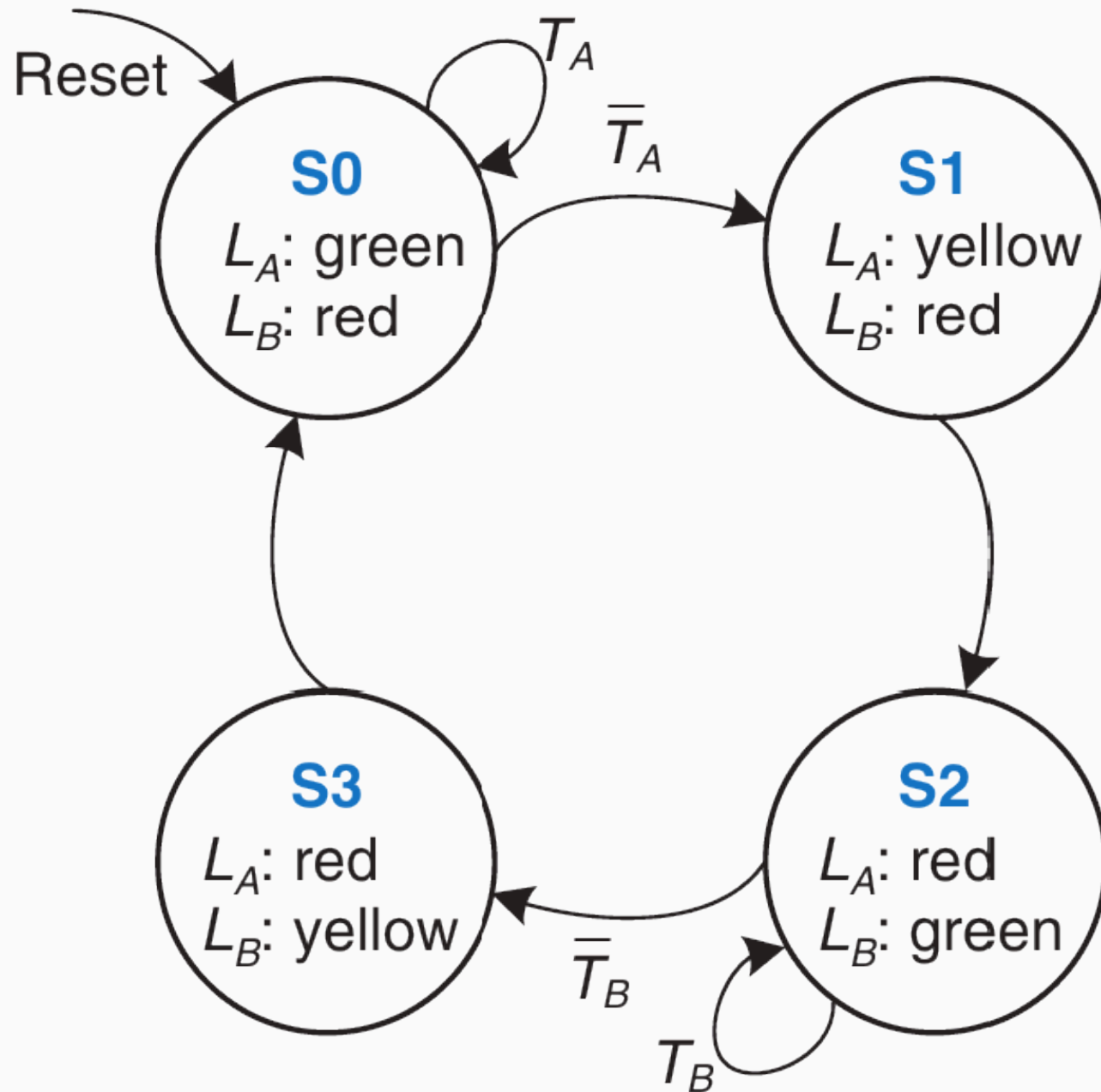


Tabla de transiciones

Estado actual S	T_a	T_b	Estado Siguiente
S0	0	—	
S0	1	—	
S1	—	—	
S2	—	0	
S2	—	1	
S3	—	—	

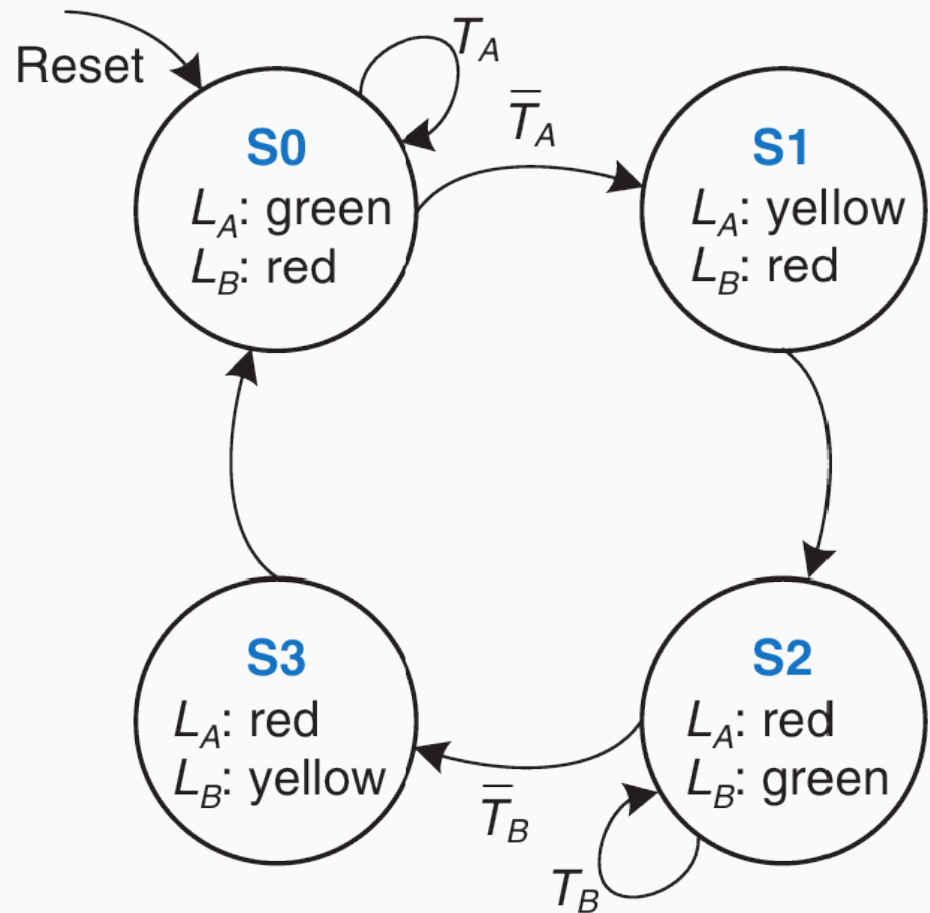


Tabla de transiciones

Estado actual S	T_a	T_b	Estado Siguiente
S0	0	–	S1
S0	1	–	
S1	–	–	
S2	–	0	
S2	–	1	
S3	–	–	

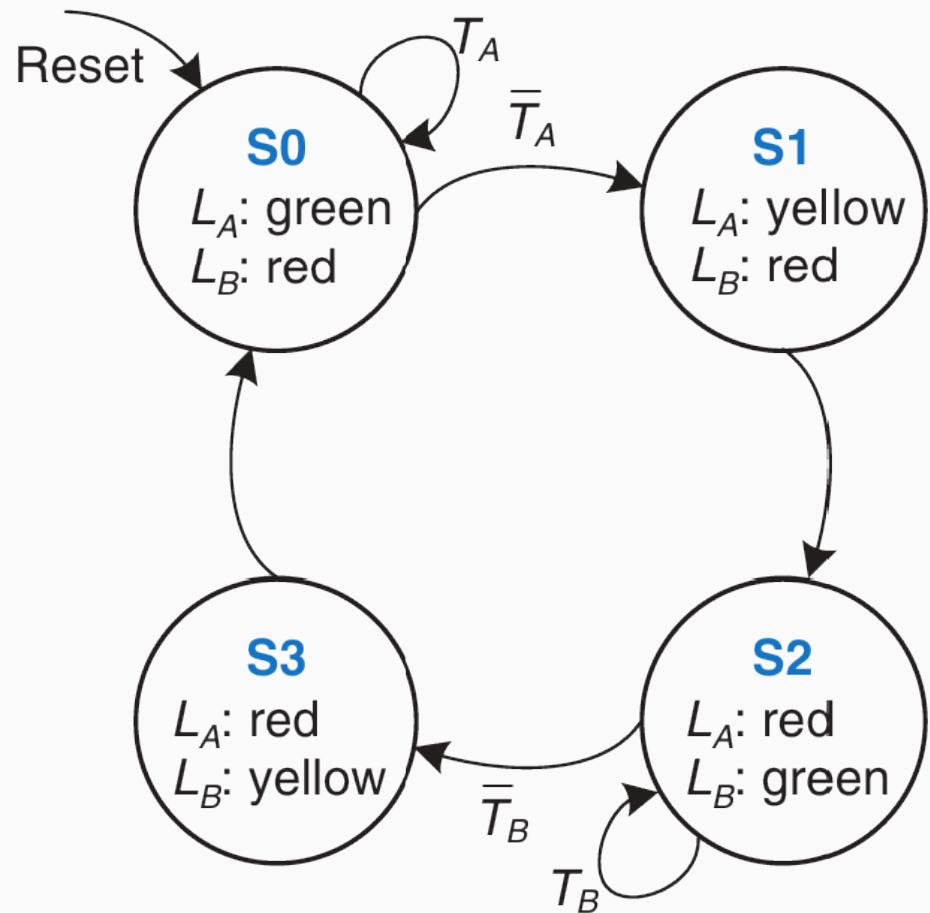


Tabla de transiciones

Estado actual S	T_a	T_b	Estado Siguiente
S0	0	–	S1
S0	1	–	S0
S1	–	–	
S2	–	0	
S2	–	1	
S3	–	–	

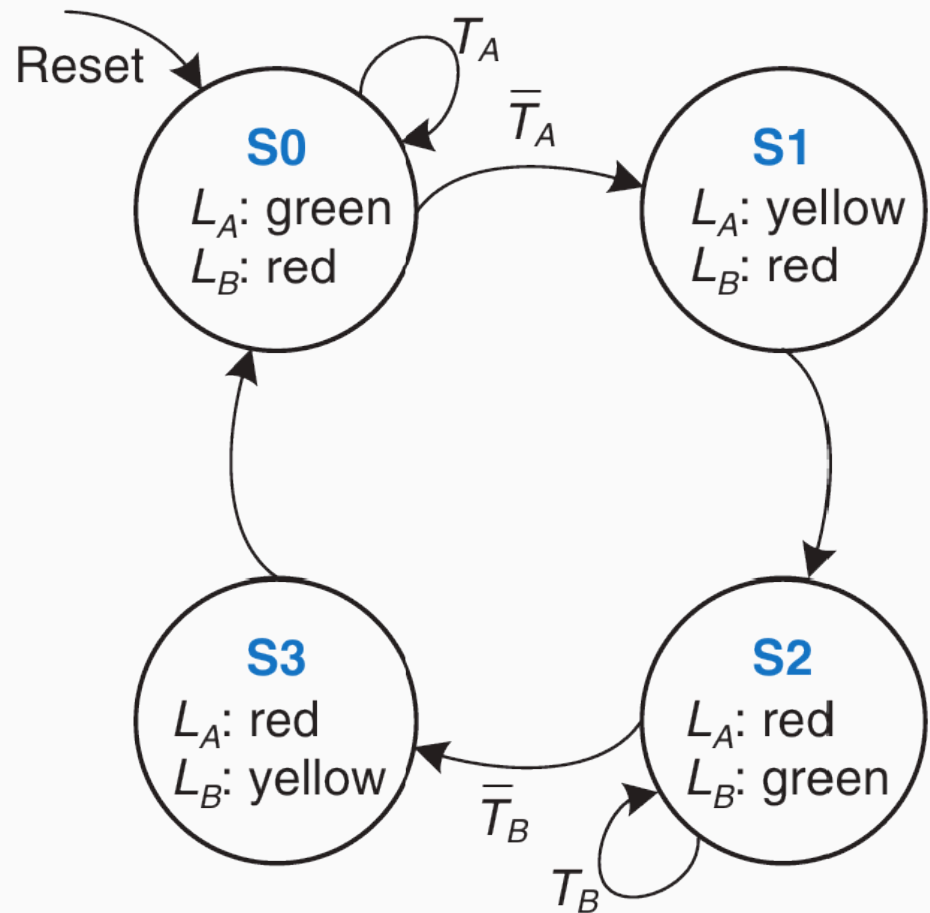


Tabla de transiciones

Estado actual S	T_a	T_b	Estado Siguiente
S0	0	–	S1
S0	1	–	S0
S1	–	–	S2
S2	–	0	
S2	–	1	
S3	–	–	

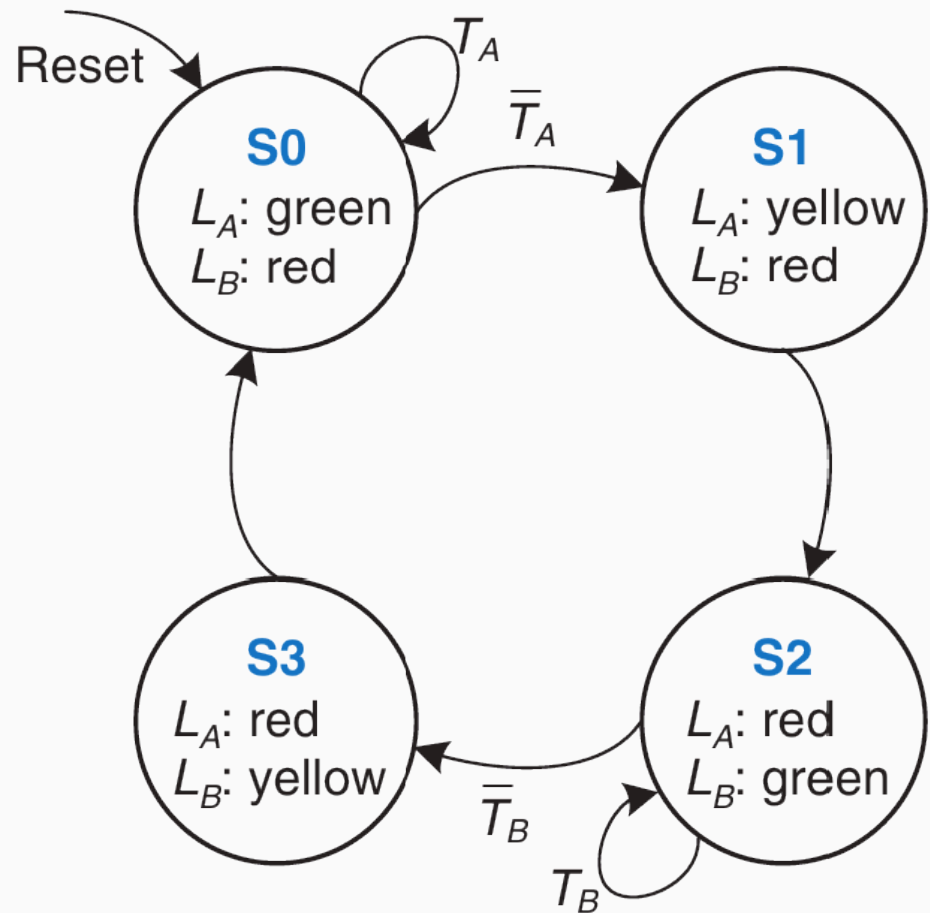


Tabla de transiciones

Estado actual S	T_a	T_b	Estado Siguiente
S0	0	–	S1
S0	1	–	S0
S1	–	–	S2
S2	–	0	S3
S2	–	1	
S3	–	–	

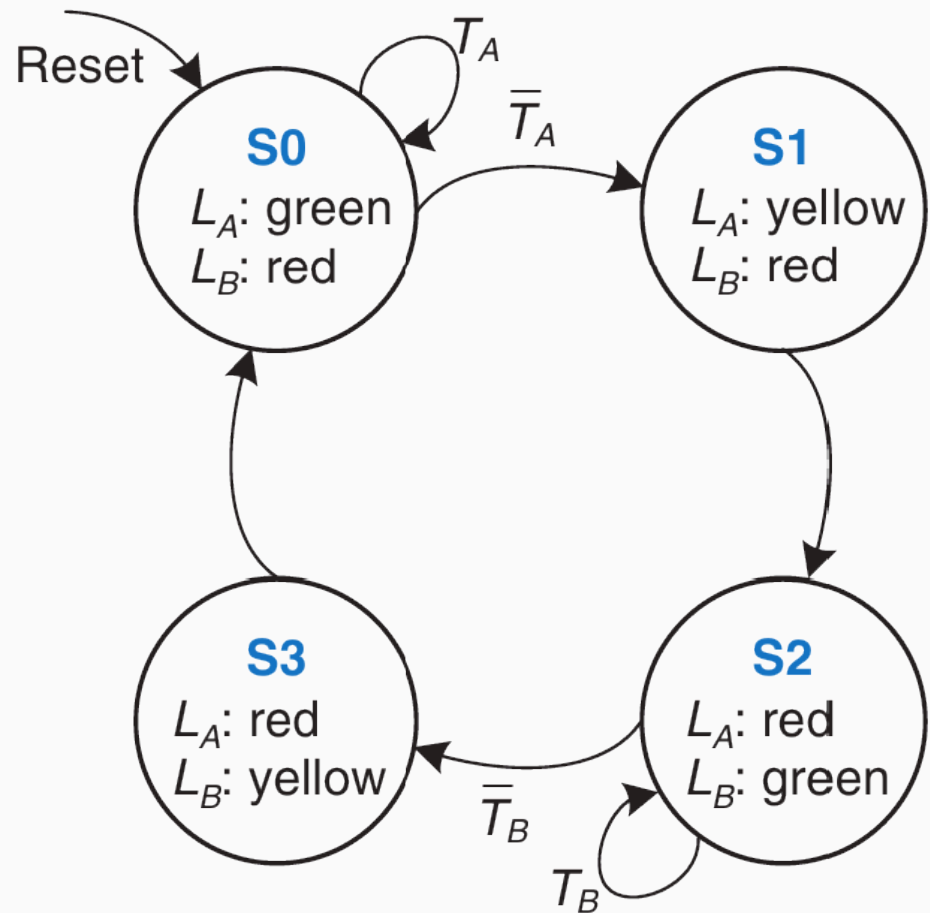


Tabla de transiciones

Estado actual S	T_a	T_b	Estado Siguiente
S0	0	–	S1
S0	1	–	S0
S1	–	–	S2
S2	–	0	S3
S2	–	1	S2
S3	–	–	

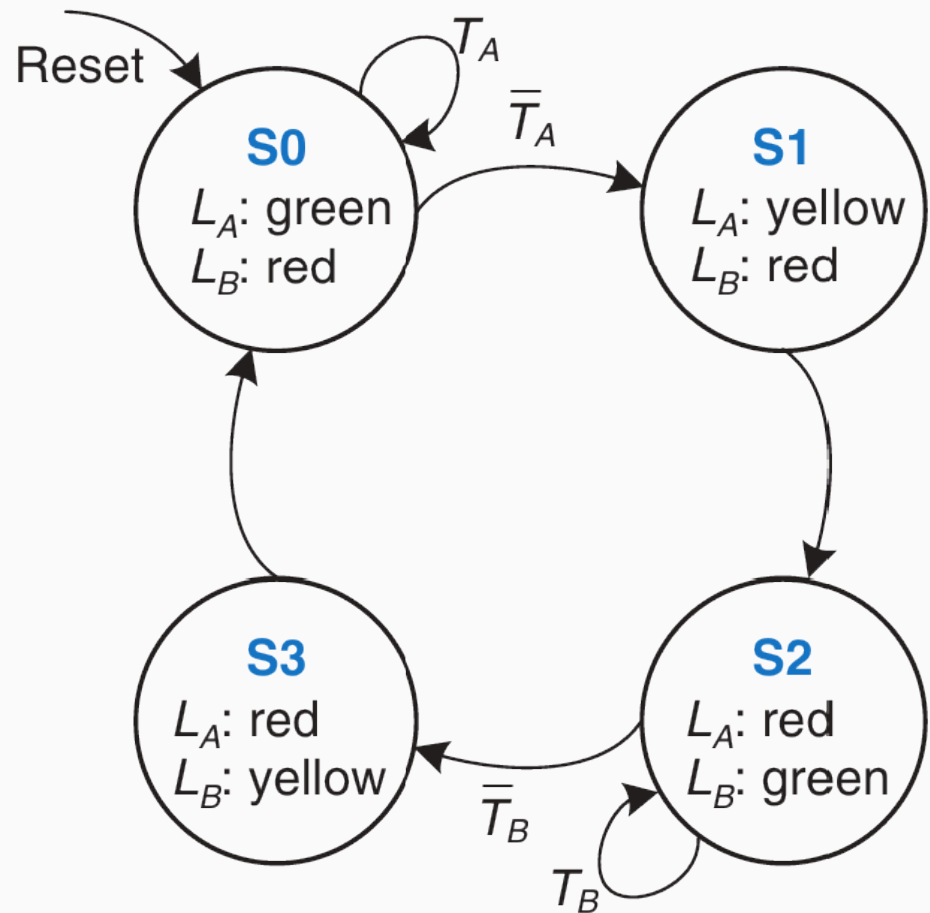


Tabla de transiciones

Estado actual S	T_a	T_b	Estado Siguiente
S0	0	–	S1
S0	1	–	S0
S1	–	–	S2
S2	–	0	S3
S2	–	1	S2
S3	–	–	S0

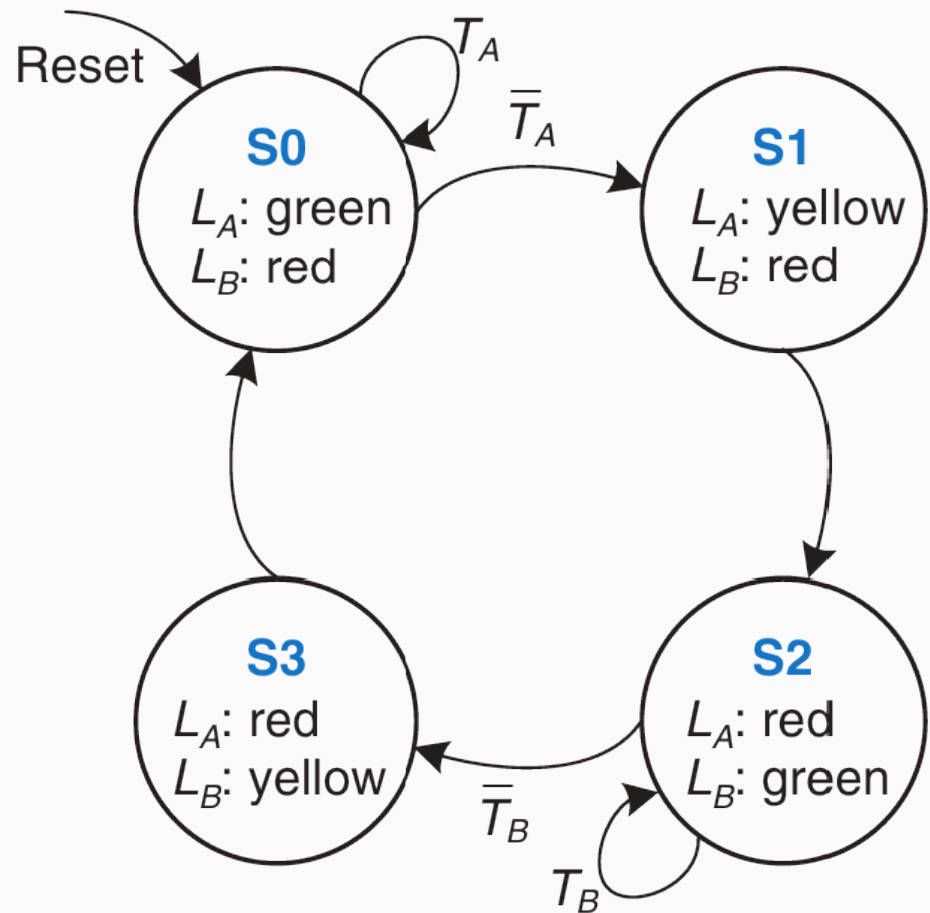


Tabla de salida

Estado actual S	L_a	L_b
S0		
S1		
S2		
S3		

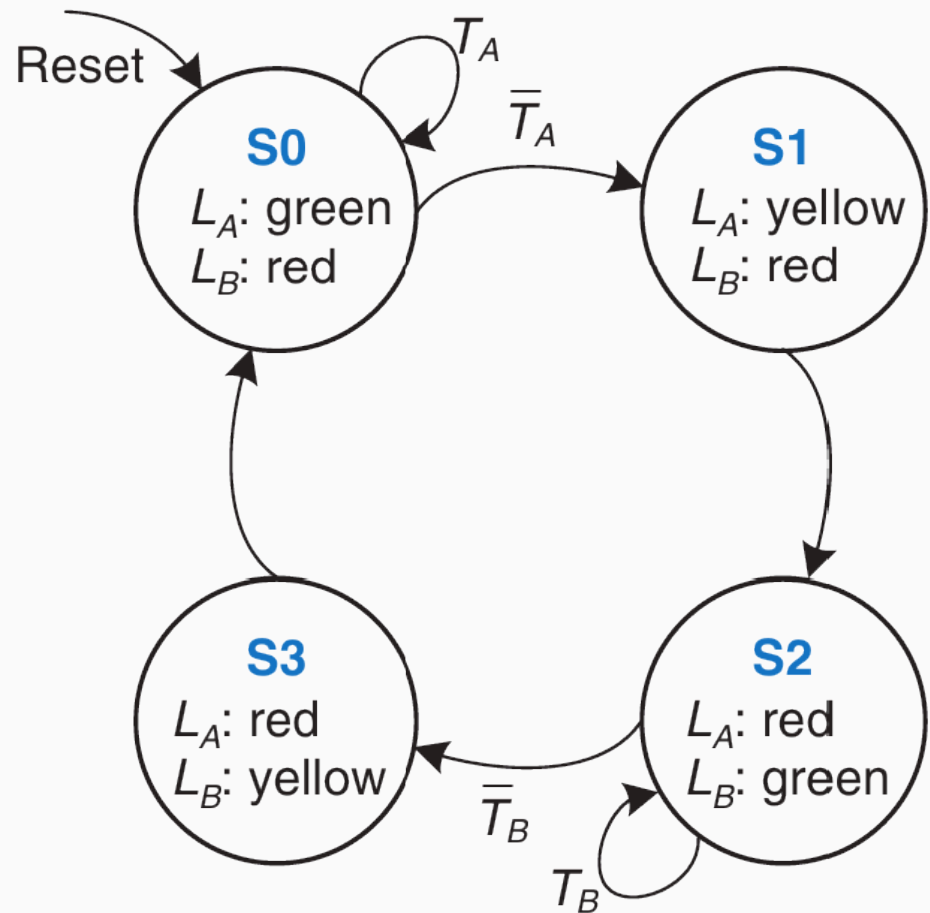


Tabla de salida

Estado actual S	L_a	L_b
S0	green	red
S1		
S2		
S3		

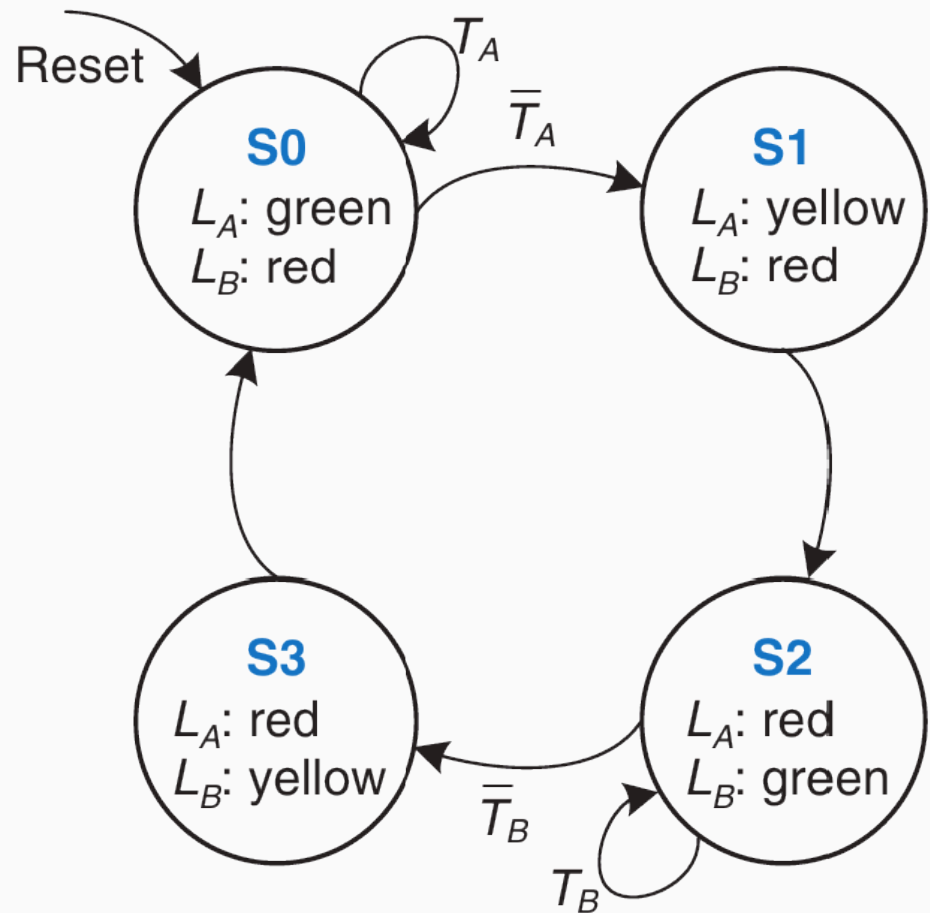


Tabla de salida

Estado actual S	L_a	L_b
S0	green	red
S1	yellow	red
S2		
S3		

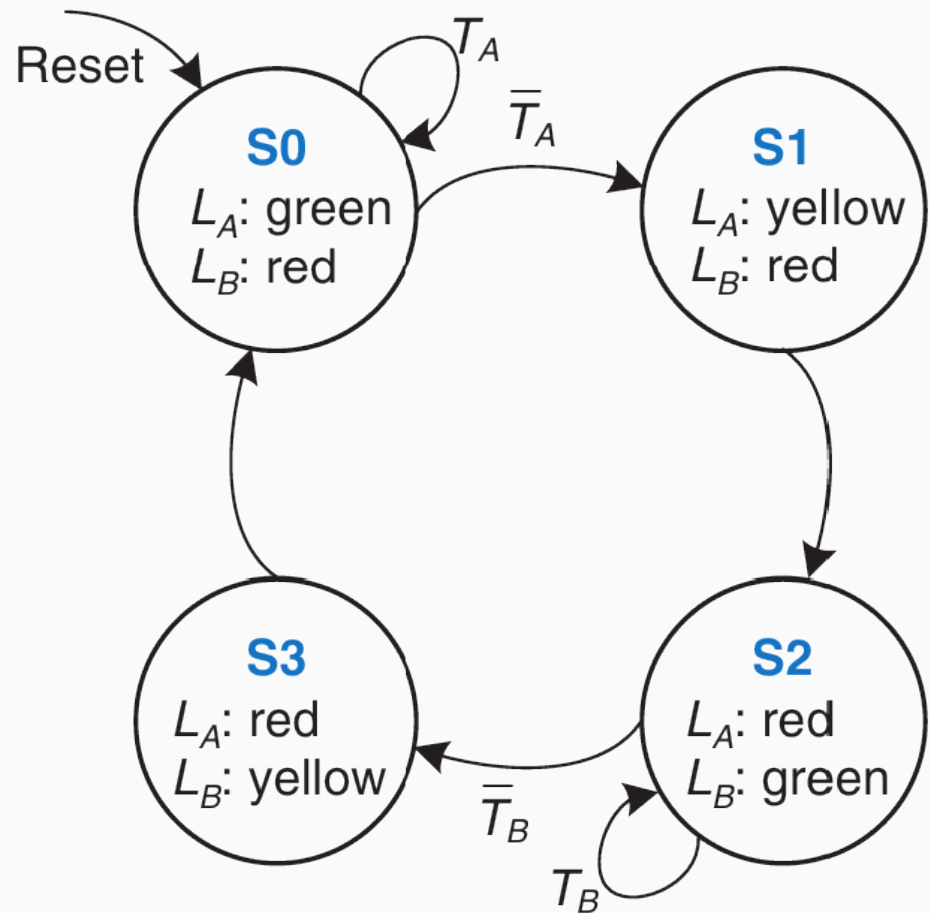


Tabla de salida

Estado actual S	L_a	L_b
S0	green	red
S1	yellow	red
S2	red	green
S3		

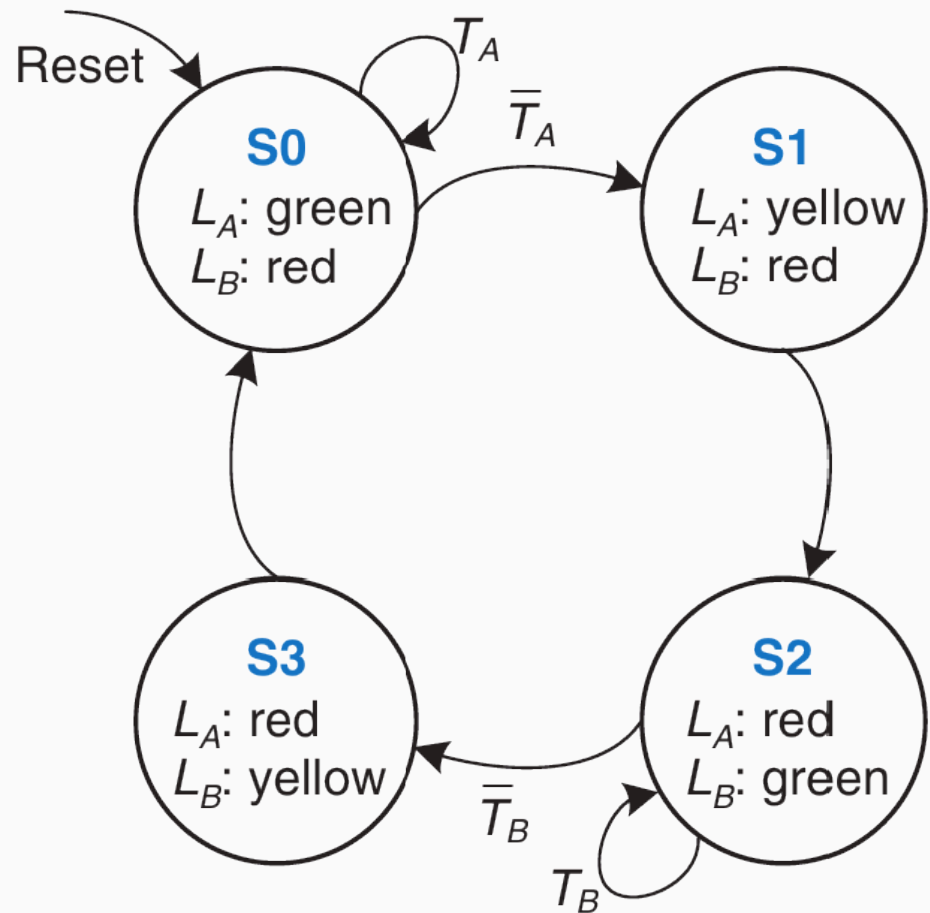
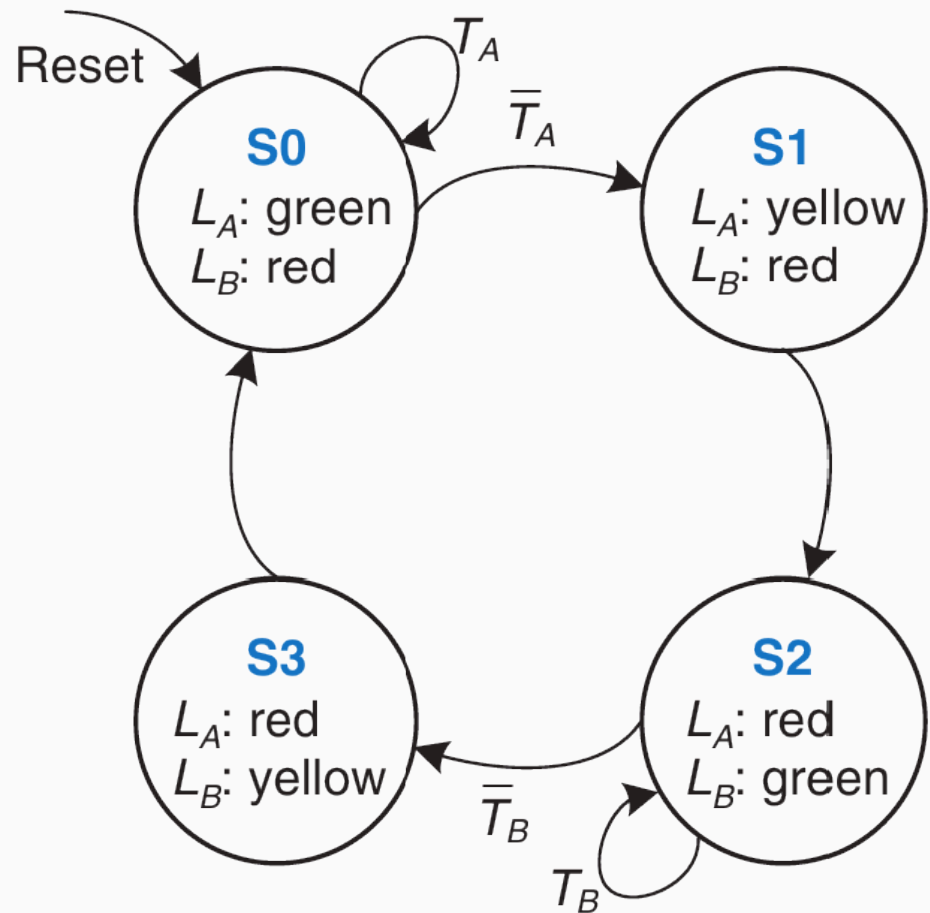


Tabla de salida

Estado actual S	L_a	L_b
S0	green	red
S1	yellow	red
S2	red	green
S3	red	yellow



Codificación

Codificación de estados

Estado actual S	Codificación $S_{1:0}$
S0	00
S1	01
S2	10
S3	11

Codificación

Codificación de estados

Estado actual S	Codificación $S_{1:0}$
S0	00
S1	01
S2	10
S3	11

Codificación de salidas

Salida	Codificación $L_{1:0}$
green	00
yellow	01
red	10

Tabla de transiciones con codificación

S	T_a	T_b	S^*
S0	0	–	S1
S0	1	–	S0
S1	–	–	S2
S2	–	0	S3
S2	–	1	S2
S3	–	–	S0

S_1	S_0	T_a	T_b	S_1^*	S_0^*
0	0	0	–	0	1
0	0	1	–	0	0
0	1	–	–	1	0
1	0	–	0	1	1
1	0	–	1	1	0
1	1	–	–	0	0

Tabla de transiciones con codificación

S	T_a	T_b	S^*
S0	0	–	S1
S0	1	–	S0
S1	–	–	S2
S2	–	0	S3
S2	–	1	S2
S3	–	–	S0

S_1	S_0	T_a	T_b	S_1^*	S_0^*
0	0	0	–	0	1
0	0	1	–	0	0
0	1	–	–	1	0
1	0	–	0	1	1
1	0	–	1	1	0
1	1	–	–	0	0

$$S_1^* = S_1 \oplus S_0$$

$$S_0^* = \overline{S_1} \cdot \overline{S_0} \cdot \overline{T_a} + S_1 \cdot \overline{S_0} \cdot \overline{T_b}$$

Tabla de salida con codificación

S	L_a	L_b
S0	green	red
S1	yellow	red
S2	red	green
S3	red	yellow

S_1	S_0	L_{a1}	L_{a0}	L_{b1}	L_{b0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

Tabla de salida con codificación

S	L_a	L_b
S0	green	red
S1	yellow	red
S2	red	green
S3	red	yellow

S_1	S_0	L_{a1}	L_{a0}	L_{b1}	L_{b0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

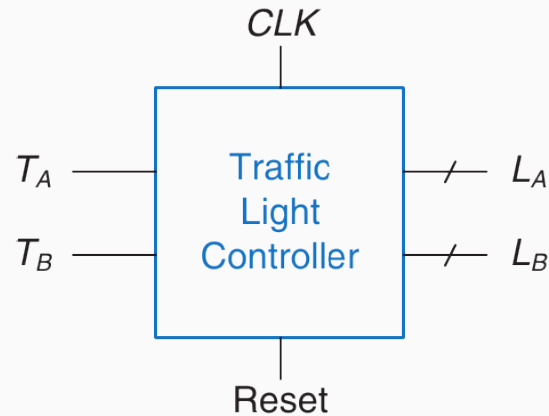
$$L_{a1} = S_1$$

$$L_{a0} = \overline{S_1} \cdot S_0$$

$$L_{b1} = \overline{S_1}$$

$$L_{b0} = S_1 \cdot S_0$$

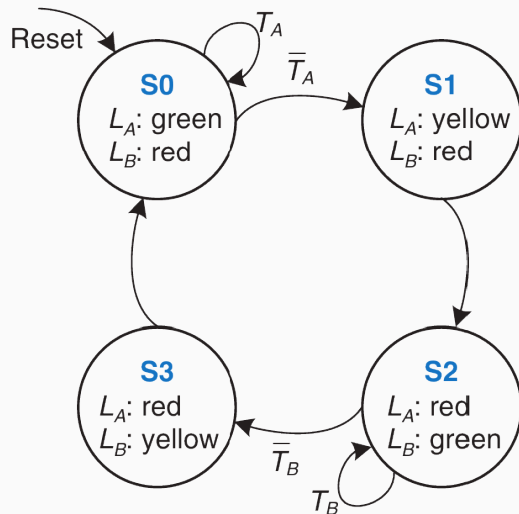
HDL al rescate!



```
module traffic_light_fsm (  
input  logic      clk,    // Reloj del sistema  
input  logic      reset, // Reset asíncrono, activo en alto → va a S0  
input  logic      T_A,    // Sensor de tránsito calle A (1 = hay vehículos)  
input  logic      T_B,    // Sensor de tránsito calle B (1 = hay vehículos)  
output logic [1:0] L_A,   // Luz del semáforo A  
output logic [1:0] L_B    // Luz del semáforo B  
);
```

SystemVerilog

Ejemplo: Controlador de luces de tráfico



// Lista de estados: mismos nombres que en el diagrama

SystemVerilog

```
typedef enum logic [1:0] {
```

```
    S0,    // L_A verde,    L_B rojo (opcional: S0 =  
           2'b00 ← Fuerza la codificación)
```

```
    S1,    // L_A amarillo, L_B rojo (opcional: S1 =  
           2'b01 ← Fuerza la codificación)
```

```
    S2,    // L_A rojo,     L_B verde (opcional: S2 =  
           2'b10 ← Fuerza la codificación)
```

```
    S3     // L_A rojo,     L_B amarillo (opcional:  
           S3 = 2'b11 ← Fuerza la codificación)
```

```
} state_t;
```

```
// 0 también:
```

```
//typedef enum {S0, S1, S2, S3} state_t;
```

```
// Pero esto genera estados de 32 bits sin decir  
nada, y es un desperdicio en este caso.
```

```
// Señales de estado actual y estado siguiente
```

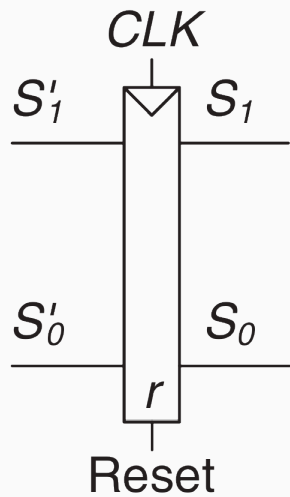
```
state_t current_state, next_state;
```

Codificación de salidas

```
localparam logic [1:0] GREEN  = 2'b00;  
localparam logic [1:0] YELLOW = 2'b01;  
localparam logic [1:0] RED    = 2'b10;
```

SystemVerilog

Registro de estado



state register

```
always_ff @(posedge clk or posedge reset) begin  
    if (reset)  
        current_state <= S0;  
    else  
        current_state <= next_state;  
end
```

SystemVerilog

Lógica de estado siguiente

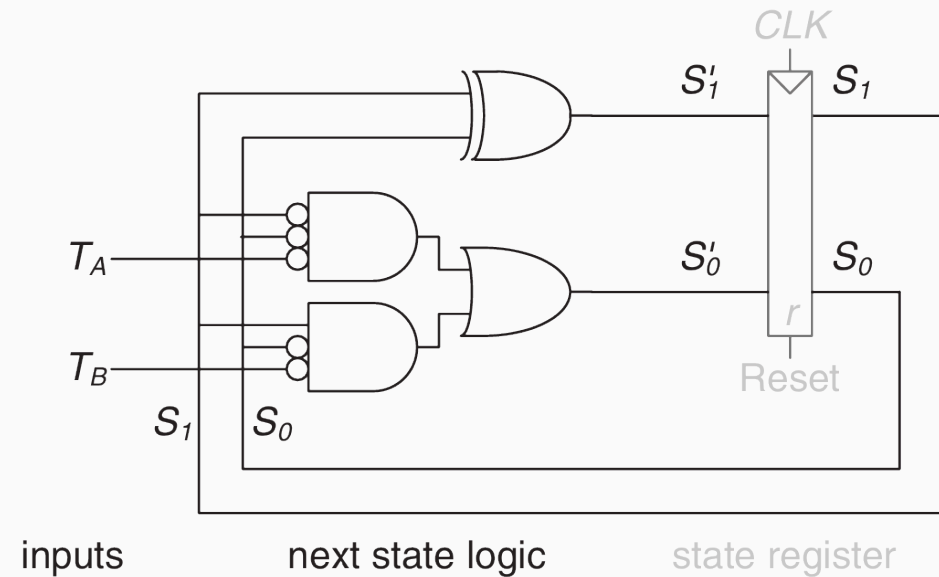
$$S_1^* = S_1 \oplus S_0$$

$$S_0^* = \overline{S_1} \cdot \overline{S_0} \cdot \overline{T_a} + S_1 \cdot \overline{S_0} \cdot \overline{T_b}$$

Lógica de estado siguiente

$$S_1^* = S_1 \oplus S_0$$

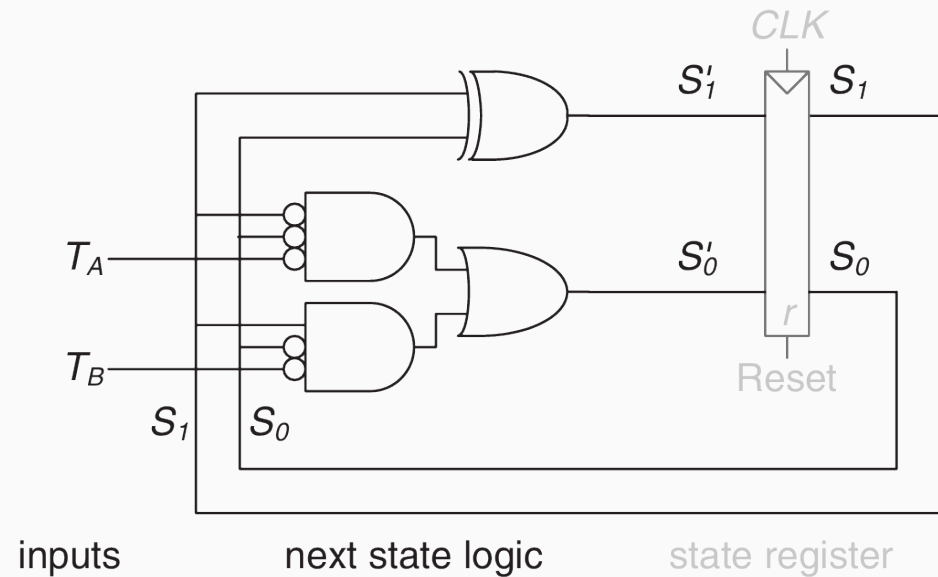
$$S_0^* = \overline{S_1} \cdot \overline{S_0} \cdot \overline{T_a} + S_1 \cdot \overline{S_0} \cdot \overline{T_b}$$



Lógica de estado siguiente

$$S_1^* = S_1 \oplus S_0$$

$$S_0^* = \overline{S_1} \cdot \overline{S_0} \cdot \overline{T_a} + S_1 \cdot \overline{S_0} \cdot \overline{T_b}$$



```
always_comb begin
    next_state = current_state; // Por defecto, me quedo donde estoy
    // continúa ...
```

SystemVerilog

Ejemplo: Controlador de luces de tráfico

```
// ... sigue always_comb
unique case (current_state)
    // S0: verde para A. Mientras el sensor T_A = 1 hay tránsito en la
    // calle A, así que el verde se mantiene (es el lazo sobre S0).
    // Cuando T_A = 0 (ya no hay tránsito en A), paso a S1.
    S0: if (!T_A) next_state = S1;

    // S1: amarillo para A, dura un solo ciclo de reloj, pasa siempre a S2.
    S1: next_state = S2;

    // S2: verde para B. Misma idea que S0 pero con el sensor T_B:
    // T_B = 1 (hay tránsito en B) → me quedo;
    // T_B = 0 (no hay más tránsito en B) → paso a S3.
    S2: if (!T_B) next_state = S3;

    // S3: amarillo para B, un ciclo y vuelvo a S0 para arrancar de nuevo.
    S3: next_state = S0;

    // Por las dudas: si caigo en un estado inválido, vuelvo a S0.
    default: next_state = S0;
endcase
end
```

SystemVerilog

Lógica de salida

$$L_{a1} = S_1$$

$$L_{a0} = \overline{S_1} \cdot S_0$$

$$L_{b1} = \overline{S_1}$$

$$L_{b0} = S_1 \cdot S_0$$

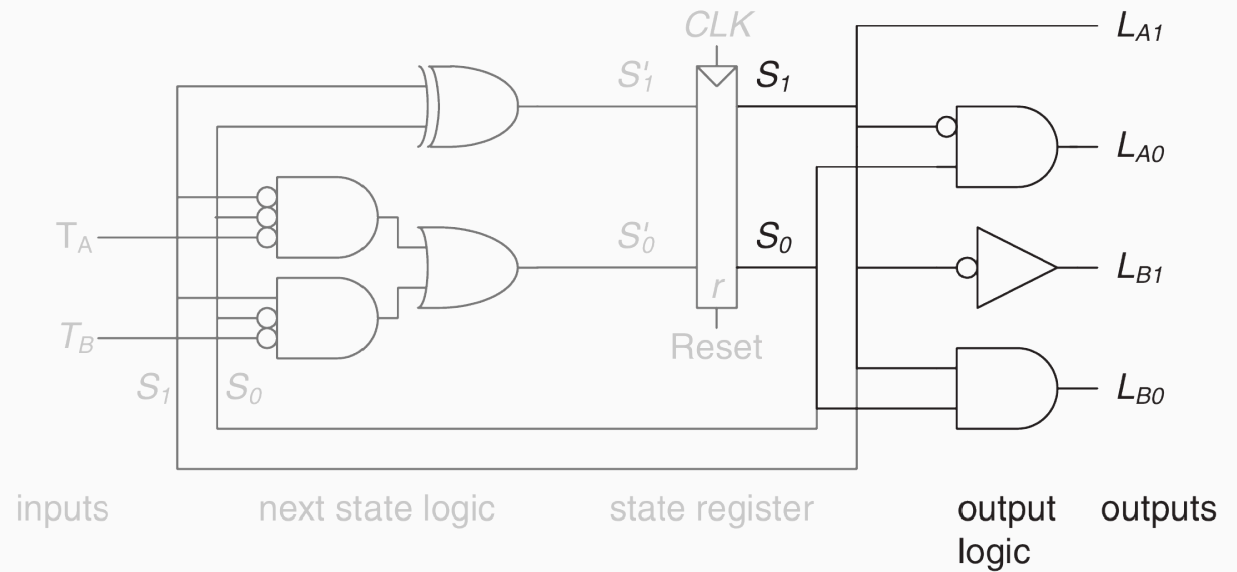
Lógica de salida

$$L_{a1} = S_1$$

$$L_{a0} = \overline{S_1} \cdot S_0$$

$$L_{b1} = \overline{S_1}$$

$$L_{b0} = S_1 \cdot S_0$$



Lógica de salida

```
always_comb begin
    unique case (current_state)
        S0: begin
            L_A = GREEN;    // Calle A en verde
            L_B = RED;      // Calle B en rojo
        end

        S1: begin
            L_A = YELLOW;   // Calle A en amarillo
            L_B = RED;      // Calle B sigue en rojo
        end

        S2: begin
            L_A = RED;      // Calle A en rojo
            L_B = GREEN;    // Calle B en verde
        end

        S3: begin
            L_A = RED;      // Calle A sigue en rojo
            L_B = YELLOW;   // Calle B en amarillo
        end
    end case
end
```

SystemVerilog

Ejemplo: Controlador de luces de tráfico

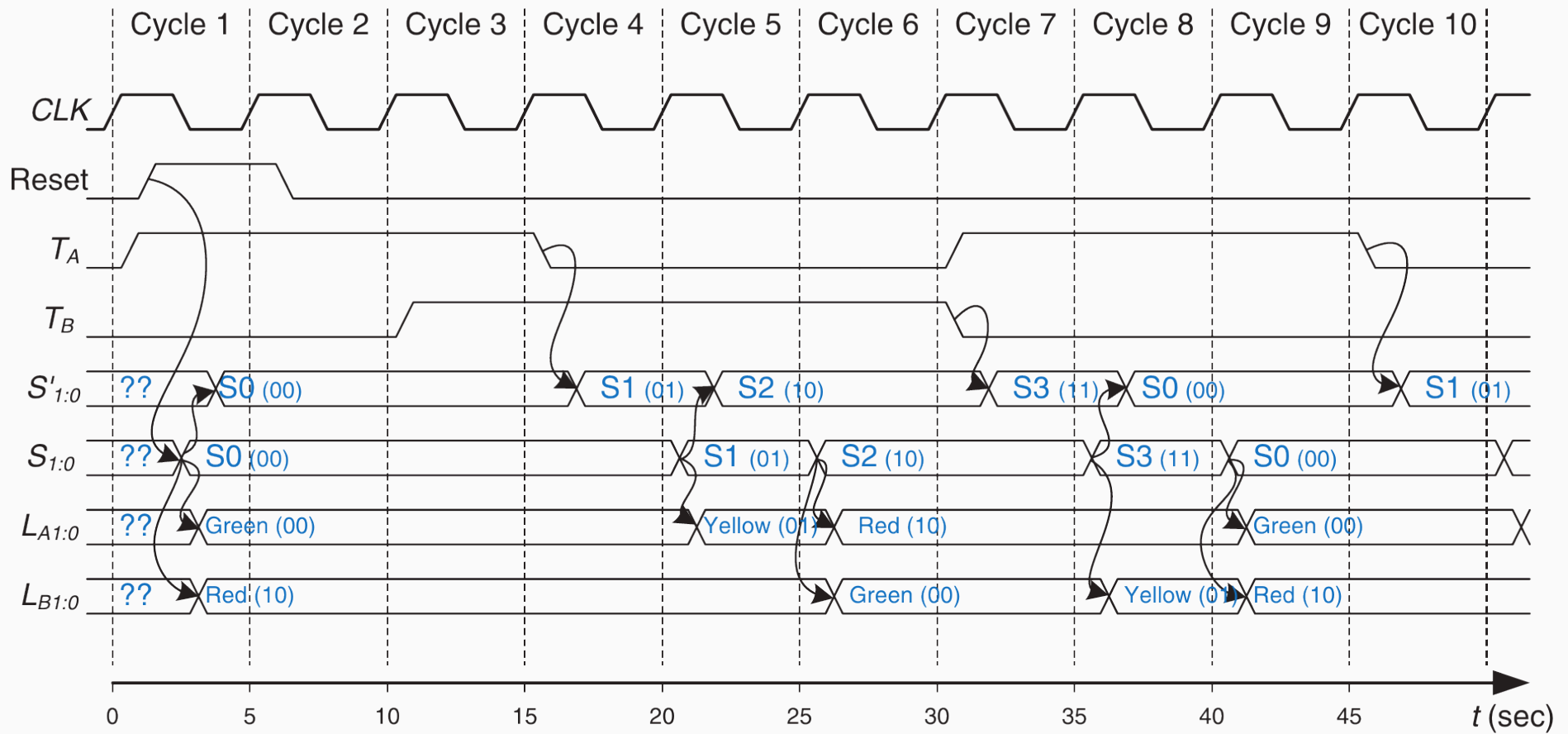
```
end

// Estado inválido: todo en rojo, que es lo más seguro.
default: begin
    L_A = RED;
    L_B = RED;
end
endcase
end

endmodule
```

Ejemplo: Controlador de luces de tráfico

Timing



Diseñar una FSM que detecte la secuencia 10 en una entrada serial de 1 bit. Cuando dicha entrada se detecte, debe prenderse una salida por un ciclo de reloj. Hacer una máquina de Moore.

Pueden seguir estos pasos:

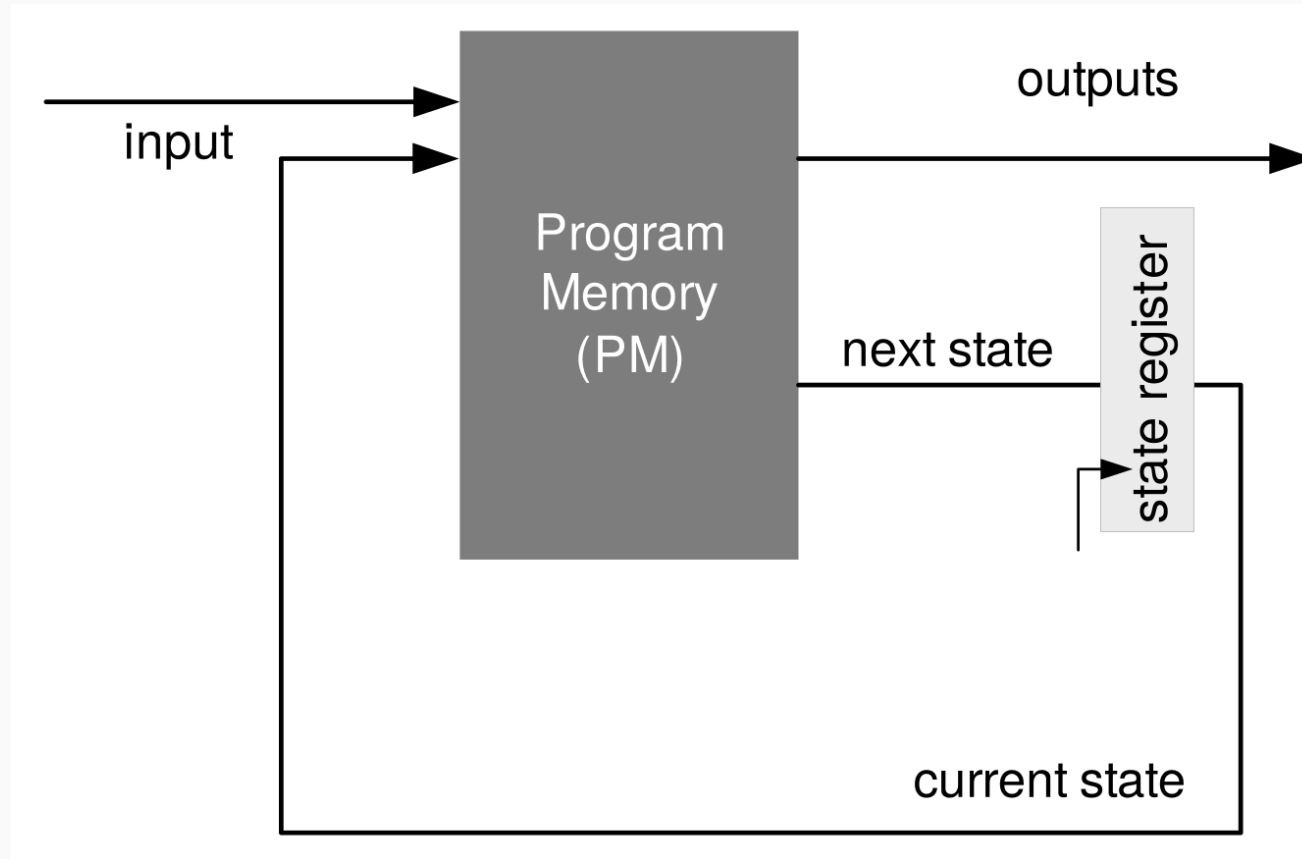
- Definir los estados y dibujar el diagrama de estados
- Hacer la tabla de transición de estados y la tabla de salida
- Codificar los estados y las salidas
- Obtener las funciones lógicas de próximo estado y de salida
- Dibujar el esquema del circuito resultante

Ahora, hacer el mismo diseño como una máquina de Mealy. ¿Qué diferencias encuentran?

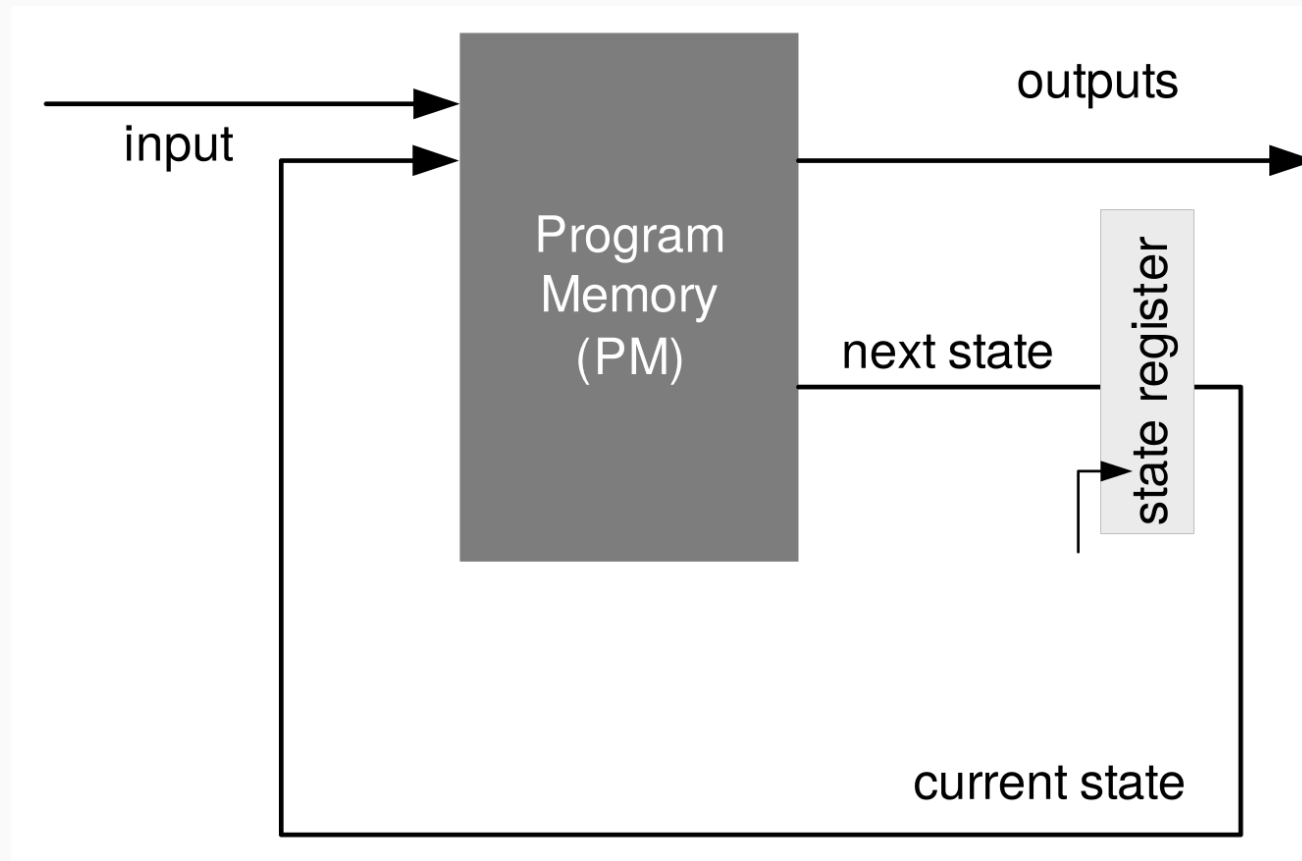
FSM Microprogramada

Ejemplo: FSM Microprogramada

Esquema general:

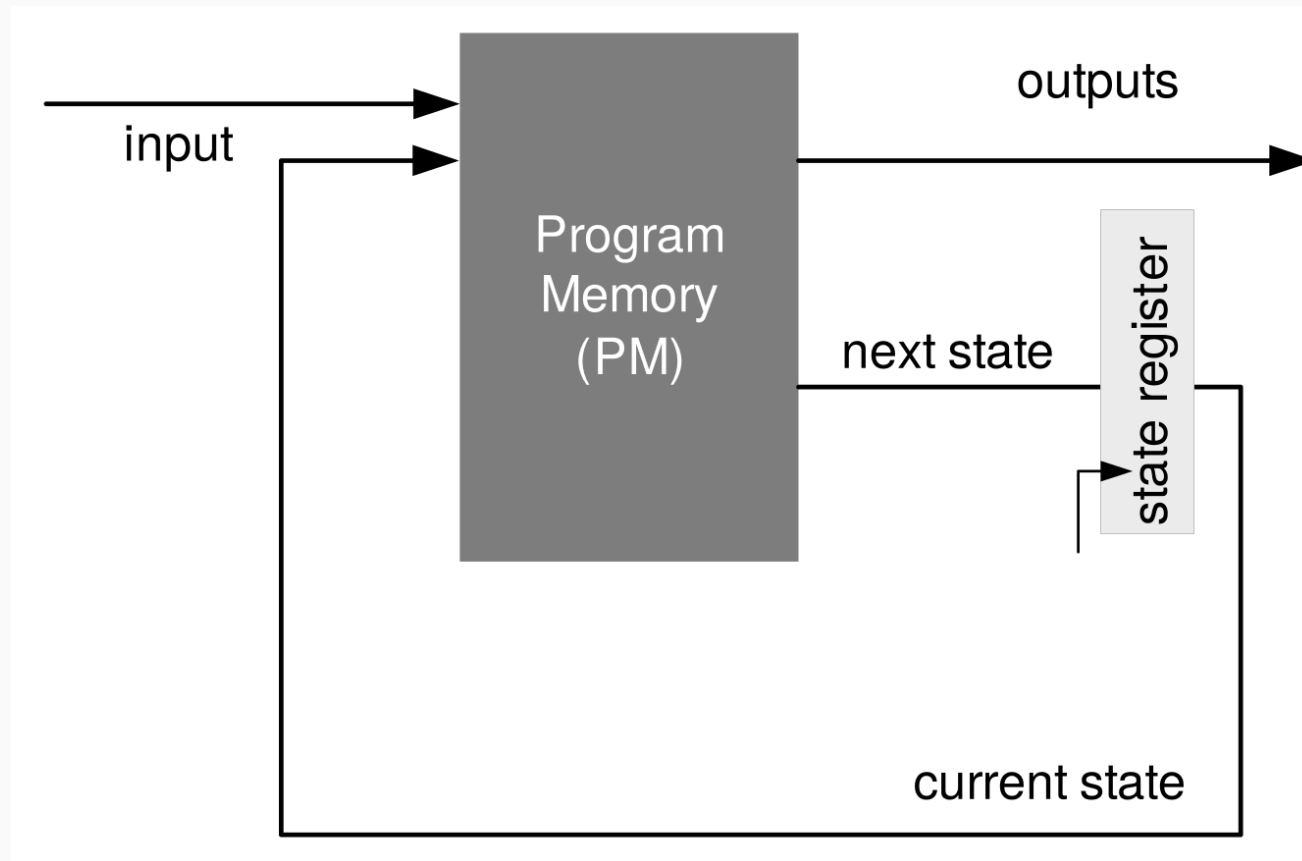


Esquema general:



- Reemplazamos la lógica combinacional *tabla de verdad* que contiene las salidas y el estado siguiente para cada combinación de estado actual y entradas.

Esquema general:



- Reemplazamos la lógica combinacional *tabla de verdad* que contiene las salidas y el estado siguiente para cada combinación de estado actual y entradas.
- Permite reconfigurar el control-path cambiando el contenido de la *memoria de programa*.

Requerimientos:

- Diseñar un circuito síncrono con **una entrada de un bit y una salida de un bit** que detecte la ocurrencia de 4 bits en 1 a la entrada. El mismo pondrá un 1 en su salida durante un ciclo de reloj luego de contar cuatro 1's. El ciclo se repite indefinidamente.
- Se puede utilizar un registro de 3 bits y una “memoria” de 3 bits y 8 posiciones.
- *¿Cómo se cambiaría el comportamiento del circuito sin cambiar el hardware?*

Resolución:

- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados:

Resolución:

- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados: 4 Codificados con

Resolución:

- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados: 4 Codificados con 2 bits (00, 01, 10, 11)

Resolución:

- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados: 4 Codificados con 2 bits (00, 01, 10, 11)
- Interfaz:
 - Lógica de salida:

Resolución:

- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados: 4 Codificados con 2 bits (00, 01, 10, 11)
- Interfaz:
 - Lógica de salida: 1 bit:

Resolución:

- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados: 4 Codificados con 2 bits (00, 01, 10, 11)
- Interfaz:
 - Lógica de salida: 1 bit: vale 1 en el último estado solamente.

Resolución:

- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados: 4 Codificados con 2 bits (00, 01, 10, 11)
- Interfaz:
 - Lógica de salida: 1 bit: vale 1 en el último estado solamente.
 - Lógica de estado siguiente:

Resolución:

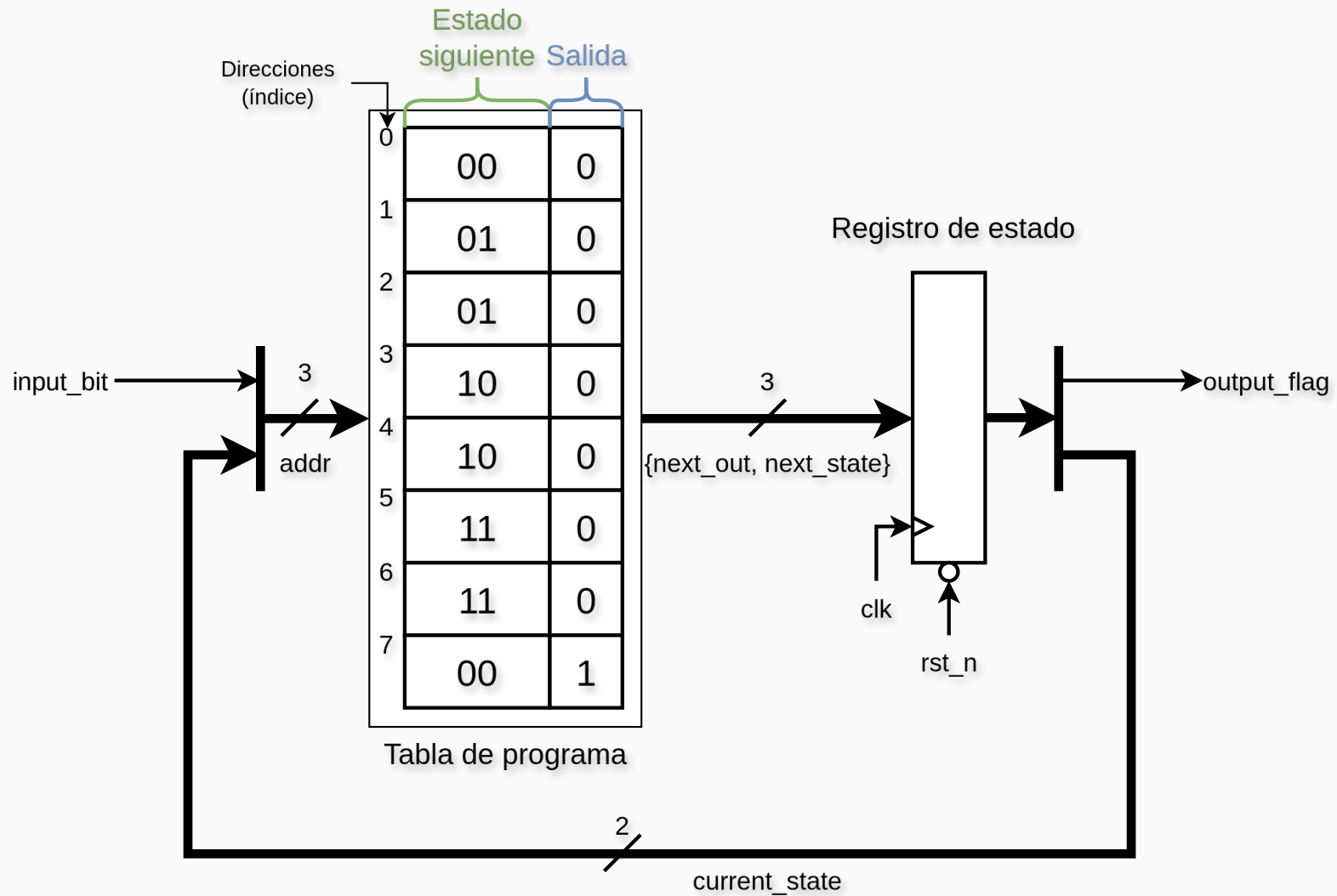
- Debemos contar hasta 4 eventos sencillos:
 - Numero de estados: 4 Codificados con 2 bits (00, 01, 10, 11)
- Interfaz:
 - Lógica de salida: 1 bit: vale 1 en el último estado solamente.
 - Lógica de estado siguiente: el estado avanza sólo cuando se recibe un 1 a la entrada.

Ejemplo: FSM Microprogramada

Esquema:

Ejemplo: FSM Microprogramada

Esquema:



Un poco (más) de HDL; concatenación de señales:

```
module mic_prog_fsm (                                     SystemVerilog
    input  logic      clk,          // Reloj del sistema
    input  logic      reset,        // Reset asíncrono, activo en alto → va a S0
    input  logic      input_bit,    // Entrada serial de un bit
    output logic      output_flag   // Salida: señal de detección
);
...

logic [2:0] addr;          // Dirección de la ROM (o índice de la tabla)
logic [1:0] current_state, next_state; // Estados, como antes

...

addr = {input_bit, current_state} // addr es de 3 bits: input_bit es el MSB,
current_state son los 2 LSBs
...

```

¿Y la tabla¹?

```
module mic_prog_rom (SystemVerilog  
    input  logic [2:0] addr_in,          // Dirección de la ROM  
    output logic [1:0] next_state,       // Salida: estado siguiente  
    output logic      next_out_flag     // Salida: señal de detección  
);  
// Continúa en la siguiente filmina  
// ...
```

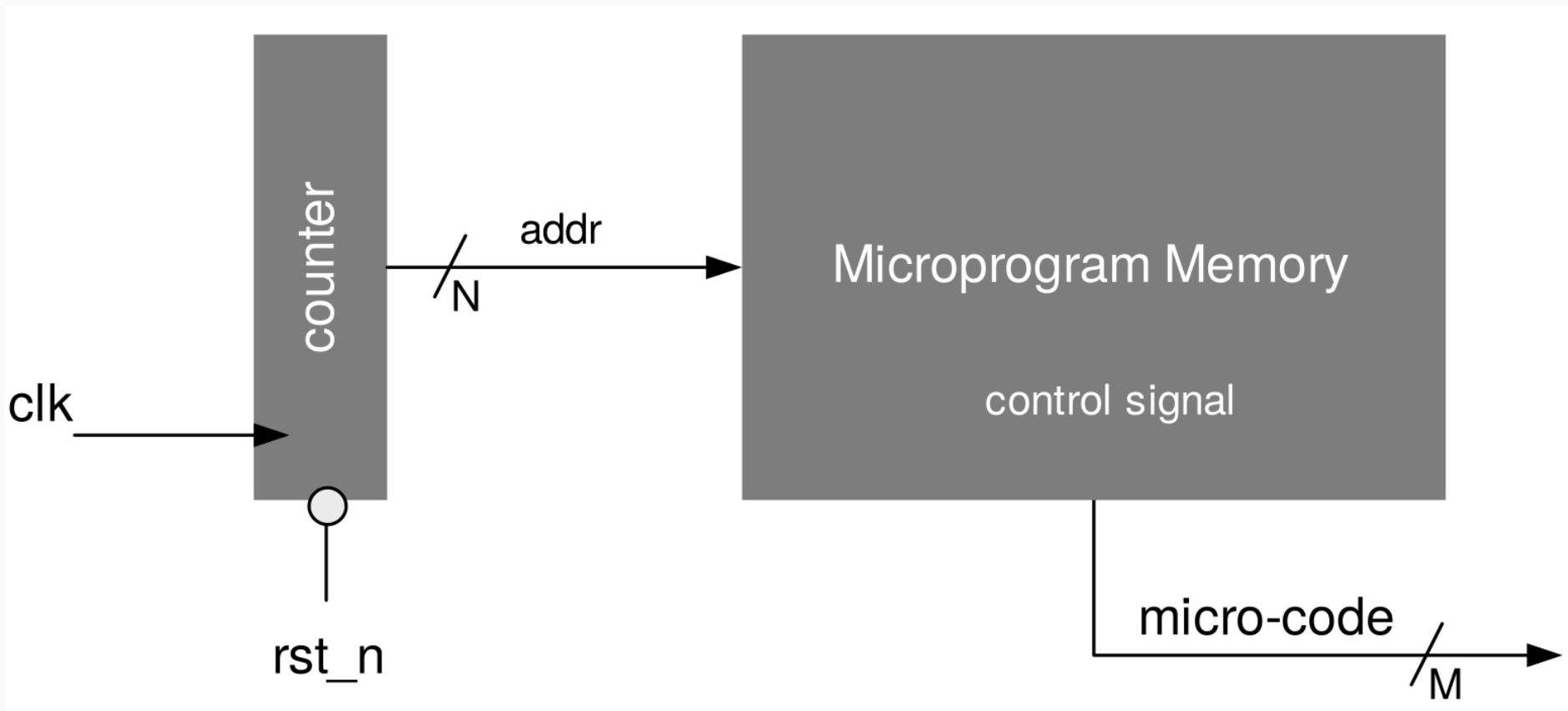
¹Llamamos tabla porque asumimos un comportamiento combinacional, una memoria propiamente dicha incluye el registro en su implementación. **En la práctica se usa una memoria directamente.**

¿Y la tabla (cont.)?

```
// Continúa de la filmina anterior
// ...
always_comb begin : ROM_b1
    // Como la tabla cubre las 8 direcciones, no hace falta valor por
    // defecto arriba; el default del case cubre los casos con X/Z.
    unique case (addr_in)
        3'b000: {next_out_flag, next_state} = {1'b0, 2'b00};
        3'b001: {next_out_flag, next_state} = {1'b0, 2'b01};
        3'b010: {next_out_flag, next_state} = {1'b0, 2'b01};
        3'b011: {next_out_flag, next_state} = {1'b0, 2'b10};
        3'b100: {next_out_flag, next_state} = {1'b0, 2'b10};
        3'b101: {next_out_flag, next_state} = {1'b0, 2'b11};
        3'b110: {next_out_flag, next_state} = {1'b0, 2'b11};
        3'b111: {next_out_flag, next_state} = {1'b1, 2'b00};
        // Por las dudas (X/Z en simulación): salida segura en cero.
        default: {next_state, four_ones_det} = {2'b00, 1'b0};
    endcase
end

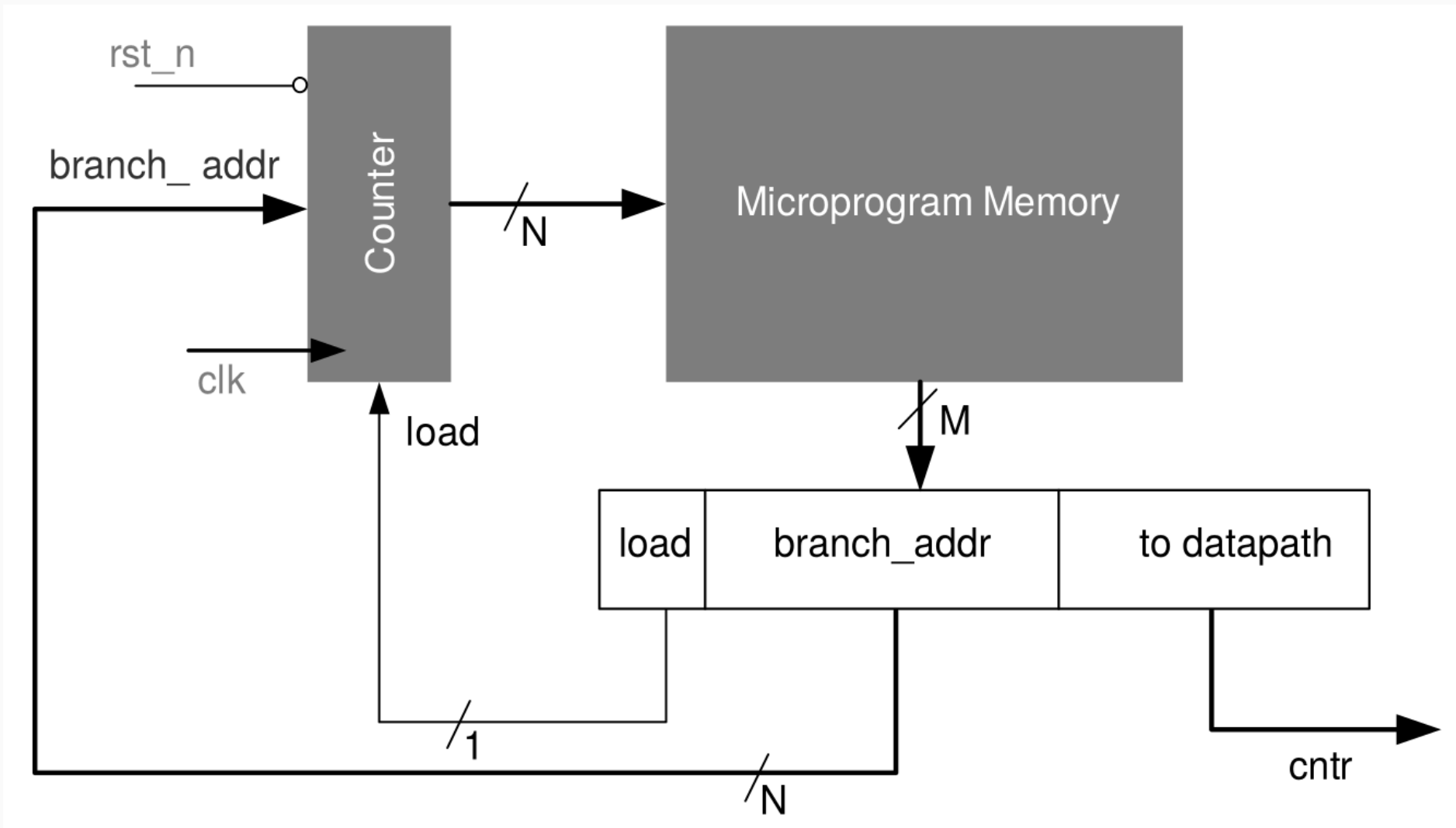
endmodule
```

Generalizando: FSMs basadas en contadores



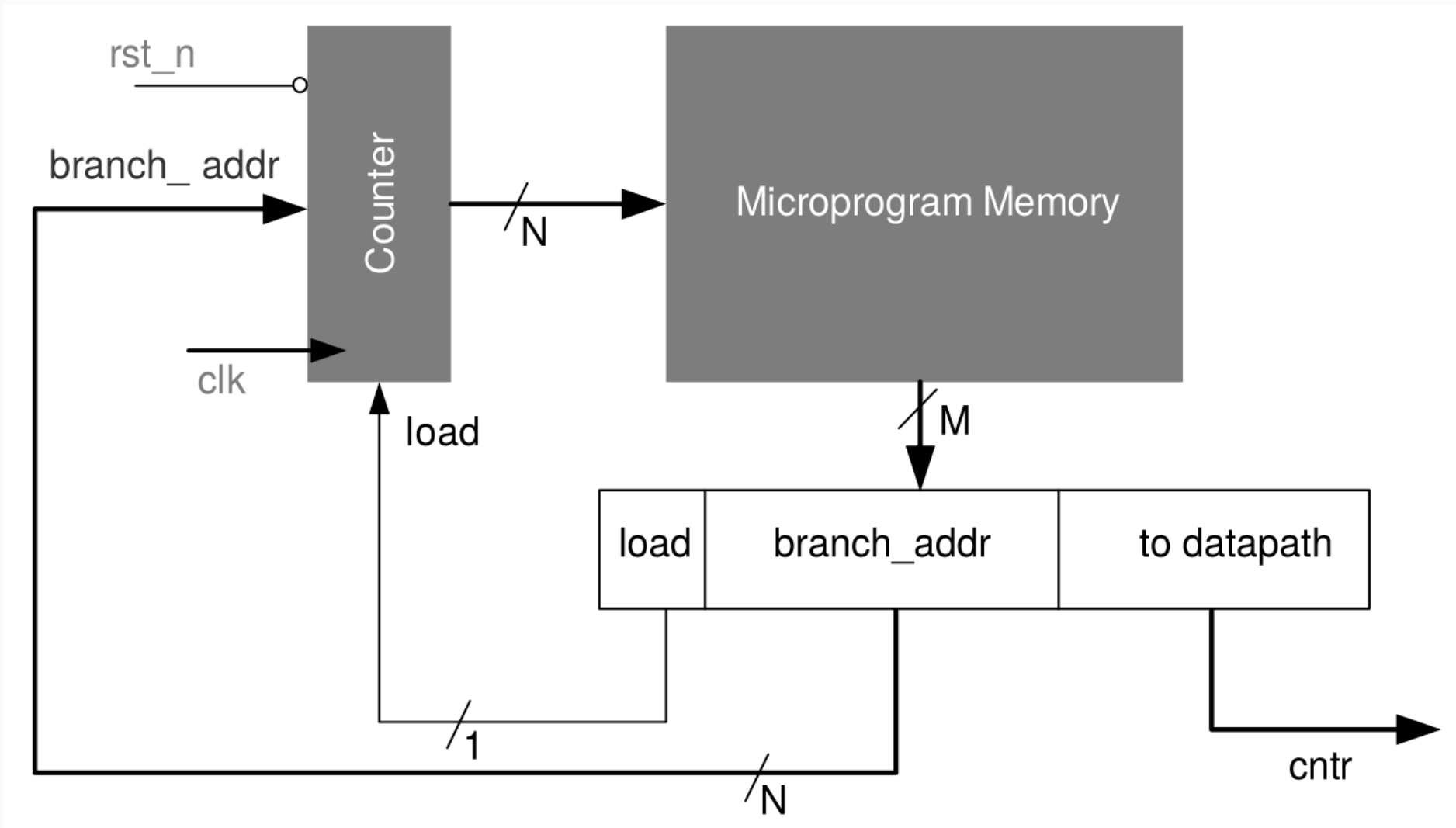
Ejemplo: FSM Microprogramada

Generalizando: FSMs basadas en contadores con carga



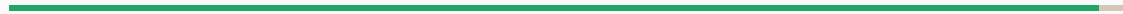
Ejemplo: FSM Microprogramada

Generalizando: FSMs basadas en contadores con carga



Unidad de control de un micro

¿Preguntas?



- Digital Design and Computer Architecture, David Harris, Sarah Harris, Morgan Kaufmann, 2012.
- Digital Design of Digital Processing Systems, Shoab Amed Khan, Springer, 2015.